Created on....11.07.2014 Last changed on/by..... 11.07.2014 SAB_VAIBHAV
Author.....SAB_VAIBHAV

Title.....Program ZFI_BNK_APP1

Type.....M Module Pool
Status.....
Application.....
Class.....

Authorization group...
Package.....ZFI_OTH

Deactivate editor lock
X Fixed point arithmetic active

```
 1
 2  *&---------------------------------------------------------------------*
 3  *& Module Pool       ZFI_BNK_APP1
 4  *&
 5  *&---------------------------------------------------------------------*
 6  *&
 7  *&
 8  *&---------------------------------------------------------------------*
 9
10
11
12
13   INCLUDE zfi_bnk_app1_top                        .    " global Data
14
15   INCLUDE zbcm_class.
16   INCLUDE zbcm_class_declare.
17
18   INCLUDE zfi_bnk_app1_o01                        .  " PBO-Modules
19   INCLUDE zfi_bnk_app1_i01                        .  " PAI-Modules
20   INCLUDE zfi_bnk_app1_f01                        .  " FORM-Routines
```

```
*&---------------------------------------------------------------------*
*&  Include           ZBCM_CLASS
*&---------------------------------------------------------------------*
CLASS lcl_file_verifier DEFINITION CREATE PRIVATE FINAL.

  PUBLIC SECTION.

    CLASS-METHODS:
      get_instance
        RETURNING
        value(ro_obj) TYPE REF TO lcl_file_verifier.

*    METHODS:
*      get_file_info " get the file data, type and other information
*        IMPORTING
*          value(iv_filename) TYPE string
*        EXPORTING
*          value(ev_signature) TYPE xstring.

    METHODS:
      verify_signature
        IMPORTING
          value(iv_signature)       TYPE xstring
        EXPORTING
          value(ev_owner)           TYPE string
          value(ev_email)           TYPE string
          value(ev_serial)          TYPE string
          value(ev_thumbprint)      TYPE string
          value(ev_validfrom)       TYPE string
          value(ev_validto)         TYPE string
          value(ev_issuer)          TYPE string
          value(ev_bindocument_out) TYPE xstring,

          get_filename
            EXPORTING
              value(ev_filename_save) TYPE string
              value(ev_file_type)     TYPE char10.

*  PRIVATE SECTION.

    CLASS-DATA: go_object TYPE REF TO lcl_file_verifier.

    METHODS:
      get_signer_info
        IMPORTING
          value(is_signerlist) TYPE ssfsigner
          EXPORTING
            value(ev_owner)       TYPE string
            value(ev_email)       TYPE string
            value(ev_serial)      TYPE string
            value(ev_thumbprint)  TYPE string
            value(ev_validfrom)   TYPE string
            value(ev_validto)     TYPE string
            value(ev_issuer)      TYPE string
            value(e_int) type i.

    DATA: mv_filename_save  TYPE string,
          mv_file_type      TYPE char10.

ENDCLASS.                    "lcl_file_verifier DEFINITION
```

```
*&---------------------------------------------------------------------*
*&  Include           ZBCM_CLASS_DECLARE
*&---------------------------------------------------------------------*


CLASS lcl_file_verifier IMPLEMENTATION.

  METHOD get_instance.
    IF lcl_file_verifier=>go_object IS INITIAL.
      CREATE OBJECT lcl_file_verifier=>go_object.
    ENDIF.

    ro_obj = lcl_file_verifier=>go_object.
  ENDMETHOD.                    "get_instance

*  METHOD get_file_info.
*
*    TYPES : BEGIN OF ty_file,
*              content TYPE sdok_sdatx,
*            END OF ty_file.
*
*    DATA: lv_filename_save  TYPE          string,
*          lv_filename       TYPE          string,
*          lv_temp           TYPE          string,
*          lv_temp1          TYPE          string,
*          lv_extn           TYPE          string,
*          lv_invalid_file   TYPE          xfeld,
*          lv_length         TYPE          i,
*          lt_data_tab       TYPE TABLE OF ty_file,
*          lv_signature      TYPE          xstring.
*
*    CLEAR lv_invalid_file.
*
*    lv_filename = iv_filename.
*    TRANSLATE lv_filename TO LOWER CASE.
*    MOVE lv_filename TO lv_temp1.
*
*    DO.
*      SPLIT lv_temp1 AT '.'
*        INTO lv_temp
*             lv_temp1.
*
*      IF lv_temp1 IS INITIAL.
*        "invalid file.
*        lv_invalid_file = abap_true.
*        EXIT.
*      ELSEIF lv_temp1 EQ 'sig'.
*        lv_extn = lv_temp.
*        EXIT.
*      ENDIF.
*    ENDDO.
*
*    IF lv_invalid_file EQ abap_true.
**       Message: Invalid file type, only signed files can be verified
*      MESSAGE text-017 TYPE 'E'.
*      RETURN.
*    ELSE.
*      " get the original filename inorder to save the data later.
*      SPLIT lv_filename AT '.sig'
*        INTO lv_filename_save lv_temp.
*
```

```
 62  *      SPLIT lv_filename_save AT '.' INTO lv_file_down lv_dummy .
 63  *      CONCATENATE lv_file_down '_verified.' lv_extn INTO lv_file_down.
 64  **      MOVE lv_filename_save TO me->mv_filename_save.
 65  *      MOVE lv_file_down TO me->mv_filename_save.
 66  *
 67  *      TRANSLATE lv_extn TO UPPER CASE.
 68  *
 69  *      CASE lv_extn.
 70  *        WHEN 'PDF'.
 71  *          me->mv_file_type = 'BIN'.
 72  *        WHEN 'TXT'.
 73  *          me->mv_file_type = 'BIN'."'ASC'.
 74  *        WHEN OTHERS.
 75  *          me->mv_file_type = 'BIN'.
 76  *      ENDCASE.
 77  *
 78  ***     Incase of signed file the GUI_UPLOAD file type would always be BI
     N.
 79  *      CALL FUNCTION 'GUI_UPLOAD'
 80  *        EXPORTING
 81  *          filename                = lv_filename
 82  *          filetype                = me->mv_file_type "'BIN'
 83  *        IMPORTING
 84  *          filelength              = lv_length
 85  *        TABLES
 86  *          data_tab                = lt_data_tab
 87  *        EXCEPTIONS
 88  *          file_open_error         = 1
 89  *          file_read_error         = 2
 90  *          no_batch                = 3
 91  *          gui_refuse_filetransfer = 4
 92  *          invalid_type            = 5
 93  *          no_authority            = 6
 94  *          unknown_error           = 7
 95  *          bad_data_format         = 8
 96  *          header_not_allowed      = 9
 97  *          separator_not_allowed   = 10
 98  *          header_too_long         = 11
 99  *          unknown_dp_error        = 12
100  *          access_denied           = 13
101  *          dp_out_of_memory        = 14
102  *          disk_full               = 15
103  *          dp_timeout              = 16
104  *          OTHERS                  = 17.
105  *
106  *      IF sy-subrc <> 0.
107  **        Message: Error while uploading the file, please contact system
     administrator
108  *        MESSAGE text-014 TYPE 'E'.
109  *      ELSE.
110  *        " convert data into xstring.
111  *        IF me->mv_file_type EQ 'BIN'.
112  *          CALL FUNCTION 'SCMS_BINARY_TO_XSTRING'
113  *            EXPORTING
114  *              input_length = lv_length
115  *            IMPORTING
116  *              buffer       = lv_signature
117  *            TABLES
118  *              binary_tab   = lt_data_tab
119  *            EXCEPTIONS
120  *              failed       = 1
```

```
121 *              OTHERS       = 2.
122 *        ELSE.
123 *          CALL FUNCTION 'SCMS_TEXT_TO_XSTRING'
124 *            IMPORTING
125 *              buffer   = lv_signature
126 *            TABLES
127 *              text_tab = lt_data_tab
128 *            EXCEPTIONS
129 *              failed   = 1
130 *              OTHERS   = 2.
131 *        ENDIF.
132 *
133 *
134 *        IF sy-subrc <> 0.
135 **         Message: Data conversion error, please contact system adminis
    trator
136 *          MESSAGE text-015 TYPE 'E'.
137 *        ELSE.
138 *          ev_signature = lv_signature.
139 *        ENDIF.
140 *
141 *      ENDIF.
142 *
143 *    ENDIF.
144 *
145 *  ENDMETHOD.                    "get_file_info
146
147   METHOD verify_signature.
148
149     DATA: lv_crc                TYPE          i,
150           lv_bindocument_out    TYPE          xstring,
151           lt_signerlist         TYPE          ssfsignertab,
152           ls_signerlist         TYPE          ssfsigner.
153
154     CALL FUNCTION 'SSFS_SERVER_VERIFY'
155       EXPORTING
156         signature        = iv_signature
157       IMPORTING
158         crc              = lv_crc
159         signerlist       = lt_signerlist
160         bindocument_out  = lv_bindocument_out
161       EXCEPTIONS
162         kernel_error     = 1
163         parameter_error  = 2
164         conversion_error = 3
165         OTHERS           = 4.
166
167     IF sy-subrc <> 0.
168 *     Message: Verification failed.
169       MESSAGE text-016 TYPE 'E'.
170     ELSE.
171       IF lv_crc <> 0 AND lv_crc <> 5.
172 *       Message: Verification failed.
173         MESSAGE text-016 TYPE 'E'.
174       ELSE.
175 *       Message: Verification Successful.
176         MESSAGE text-004 TYPE 'S'.
177         " get signer info.
178         READ TABLE lt_signerlist INTO ls_signerlist INDEX 1.
179
180         CALL METHOD get_signer_info
```

```
181             EXPORTING
182               is_signerlist = ls_signerlist
183             IMPORTING
184               ev_owner      = ev_owner
185               ev_email      = ev_email
186               ev_serial     = ev_serial
187               ev_thumbprint = ev_thumbprint
188               ev_validfrom  = ev_validfrom
189               ev_validto    = ev_validto
190               ev_issuer     = ev_issuer.
191
192           ev_bindocument_out = lv_bindocument_out.
193
194         ENDIF.
195       ENDIF.
196
197     ENDMETHOD.                    "verify_signature
198
199     METHOD get_signer_info.
200
201       DATA: lo_pdo_dig_sig  TYPE REF TO cl_abap_x509_certificate,
202             lv_text         TYPE c LENGTH 200,
203             ls_email        TYPE sx_address.
204
205       lo_pdo_dig_sig = cl_abap_x509_certificate=>get_instance( is_signerli
    st-cert ).
206
207       TRY.
208           CALL METHOD lo_pdo_dig_sig->get_subject_issuer
209             IMPORTING
210               ef_subject = ev_owner
211               ef_issuer  = ev_issuer.
212         CATCH cx_abap_x509_certificate .
213       ENDTRY.
214
215       TRY.
216           CALL METHOD lo_pdo_dig_sig->get_serial_hex
217             RECEIVING
218               rf_value = ev_serial.
219         CATCH cx_abap_x509_certificate .
220       ENDTRY.
221
222       TRY.
223           CALL METHOD lo_pdo_dig_sig->get_pk_finger_print
224             RECEIVING
225               rf_value = ev_thumbprint.
226         CATCH cx_abap_x509_certificate .
227       ENDTRY.
228
229       TRY.
230           CALL METHOD lo_pdo_dig_sig->get_valid_from
231             IMPORTING
232               ef_gmt_string = ev_validfrom.
233         CATCH cx_abap_x509_certificate .
234       ENDTRY.
235
236       TRY.
237           CALL METHOD lo_pdo_dig_sig->get_valid_to
238             IMPORTING
239               ef_gmt_string = ev_validto.
240         CATCH cx_abap_x509_certificate .
```

```
241      ENDTRY.
242
243
244  try.
245
246  *  TRY.
247    CALL METHOD lo_pdo_dig_sig->get_version
248      RECEIVING
249        rf_int = e_int
250        .
251  *   CATCH cx_abap_x509_certificate .
252    ENDTRY.
253
254
255  *CALL METHOD lo_pdo_dig_sig->GET_VERSION
256  *returning
257  *  rf_int = e_int.
258  *endtry.
259
260      TRY.
261          CALL METHOD lo_pdo_dig_sig->get_subject_alt_string
262            RECEIVING
263              rf_string = ev_email.
264
265          IF ev_email IS NOT INITIAL.
266            CLEAR lv_text.
267            SPLIT ev_email AT '=' INTO ev_email lv_text.
268
269            ls_email-type = 'INT'.
270            ls_email-address = lv_text.
271
272            CALL FUNCTION 'SX_INTERNET_ADDRESS_TO_NORMAL'
273              EXPORTING
274                address_unstruct    = ls_email
275              EXCEPTIONS
276                error_address_type  = 1
277                error_address       = 2
278                error_group_address = 3
279                OTHERS              = 4.
280            IF sy-subrc = 0.
281              ev_email = lv_text.
282            ELSE.
283              CLEAR ev_email.
284            ENDIF.
285          ENDIF.
286
287        CATCH cx_abap_x509_certificate .
288      ENDTRY.
289
290
291    ENDMETHOD.                    "get_signer_info
292
293
294    METHOD get_filename.
295
296      ev_filename_save = mv_filename_save.
297      ev_file_type = mv_file_type.
298
299    ENDMETHOD.                    "get_filename
300
301  ENDCLASS.                     "lcl_utility IMPLEMENTATION
```

```
*&---------------------------------------------------------------------*
*&  Include           ZFI_BNK_APP1_F01
*&---------------------------------------------------------------------*

*&---------------------------------------------------------------------*
*&      Form  F_PREPARE_OP_TAB1
*&---------------------------------------------------------------------*
FORM f_prepare_op_tab1 .

  REFRESH : gt_paym, gt_batch_header,gt_batch_sign,gt_final.
  SELECT * FROM zfi_paym_file INTO TABLE gt_paym.

  DELETE gt_paym WHERE file_data_sent IS NOT INITIAL.
  DELETE gt_paym WHERE raw_data IS INITIAL.

  IF gt_paym IS NOT INITIAL.
    SELECT * FROM bnk_batch_header INTO TABLE gt_batch_header
                          FOR ALL ENTRIES IN gt_paym
                          WHERE laufi_f = gt_paym-laufi
                          AND   laufd_f = gt_paym-laufd.

    IF gt_batch_header IS NOT INITIAL.
    SELECT * FROM  zfi_batch_sign INTO TABLE gt_batch_sign
                    FOR ALL ENTRIES IN gt_batch_header
                     WHERE batch_no = gt_batch_header-batch_no
                       AND    signer = sy-uname
                       and    snro  = '1'.


sort gt_batch_sign by CDATE1 CTIME1.
  LOOP AT gt_batch_header INTO gs_batch_header.
  READ TABLE gt_batch_sign INTO gs_batch_sign WITH KEY batch_no = gs_bat
ch_header-batch_no
                                                       signer = sy-unam
e
                                                        snro  = '1'.
   IF sy-subrc EQ 0 .
*    if  gs_batch_sign-signer = sy-uname   and gs_batch_sign-digitl_sign
 = ' '.
    MOVE-CORRESPONDING gs_batch_header TO gs_final.
    READ TABLE gt_paym INTO gs_paym WITH KEY laufd = gs_batch_header-lau
fd_f
                                            laufi = gs_batch_header-lau
fi_f.
    IF sy-subrc = 0.
      gs_final-raw_data = gs_paym-raw_data.
    ENDIF.
    APPEND gs_final TO gt_final.
  ENDIF.
* endif.
    CLEAR : gs_batch_header,gs_batch_sign,gs_final.
  ENDLOOP.
 ENDIF.
  ENDIF.

ENDFORM.                    " F_PREPARE_OP_TAB1

*&---------------------------------------------------------------------*
*&      Form  F_ALV_BUILD_FIELDCAT1
*&---------------------------------------------------------------------*
*       text
```

```
*---------------------------------------------------------------------*
*  ---->  p1        text
*  <----  p2        text
*---------------------------------------------------------------------*
FORM f_alv_build_fieldcat1 .
  DATA lv_fldcat1 TYPE lvc_s_fcat.
  DATA: lv_index1 TYPE sy-index.
  REFRESH : it_fcat1.
  CLEAR lv_fldcat1.
  lv_fldcat1-tabname   = 'GT_FINAL'.
  lv_fldcat1-fieldname = 'BATCH_NO'.
  lv_fldcat1-scrtext_m = 'Batch no.'.
  lv_fldcat1-outputlen = '10'.
*  lv_fldcat1-row_pos   = '1'.
  lv_fldcat1-col_pos   = lv_index1.
  APPEND lv_fldcat1 TO it_fcat1.

  lv_index1  = lv_index1 + 1.
  CLEAR lv_fldcat1.
  lv_fldcat1-tabname   = 'GT_FINAL'.
  lv_fldcat1-fieldname = 'RULE_ID'.
  lv_fldcat1-scrtext_m = 'Rule Id'.
  lv_fldcat1-outputlen = '10'.
*  lv_fldcat1-row_pos   = '1'.
  lv_fldcat1-col_pos   = lv_index1.
  APPEND lv_fldcat1 TO it_fcat1.

  lv_index1  = lv_index1 + 1.
  CLEAR lv_fldcat1.
  lv_fldcat1-tabname   = 'GT_FINAL'.
  lv_fldcat1-fieldname = 'ITEM_CNT'.
  lv_fldcat1-scrtext_m = 'No. of Payment'.
  lv_fldcat1-outputlen = '14'.
*  lv_fldcat1-row_pos   = '1'.
  lv_fldcat1-col_pos   = lv_index1.
  APPEND lv_fldcat1 TO it_fcat1.

  lv_index1  = lv_index1 + 1.
  CLEAR lv_fldcat1.
  lv_fldcat1-tabname   = 'GT_FINAL'.
  lv_fldcat1-fieldname = 'LAUFD'.
  lv_fldcat1-scrtext_m = 'Merger date'.
  lv_fldcat1-outputlen = '11'.
*  lv_fldcat1-row_pos   = '1'.
  lv_fldcat1-col_pos   = lv_index1.
  APPEND lv_fldcat1 TO it_fcat1.

  lv_index1  = lv_index1 + 1.
  CLEAR lv_fldcat1.
  lv_fldcat1-tabname   = 'GT_FINAL'.
  lv_fldcat1-fieldname = 'LAUFI'.
  lv_fldcat1-scrtext_m = 'Merge Id'.
  lv_fldcat1-outputlen = '8'.
*  lv_fldcat1-row_pos   = '1'.
  lv_fldcat1-col_pos   = lv_index1.
  APPEND lv_fldcat1 TO it_fcat1.

  lv_index1  = lv_index1 + 1.
  CLEAR lv_fldcat1.
  lv_fldcat1-tabname   = 'GT_FINAL'.
  lv_fldcat1-fieldname = 'LAUFD_F'.
```

```
118    lv_fldcat1-scrtext_m = 'File Date'.
119    lv_fldcat1-outputlen = '9'.
120  *  lv_fldcat1-row_pos   = '1'.
121    lv_fldcat1-col_pos   = lv_index1.
122    APPEND lv_fldcat1 TO it_fcat1.
123
124    lv_index1  = lv_index1 + 1.
125    CLEAR lv_fldcat1.
126    lv_fldcat1-tabname   = 'GT_FINAL'.
127    lv_fldcat1-fieldname = 'LAUFI_F'.
128    lv_fldcat1-scrtext_m = 'File Id'.
129    lv_fldcat1-outputlen = '7'.
130  *  lv_fldcat1-row_pos   = '1'.
131    lv_fldcat1-col_pos   = lv_index1.
132    APPEND lv_fldcat1 TO it_fcat1.
133
134    lv_index1  = lv_index1 + 1.
135    CLEAR lv_fldcat1.
136    lv_fldcat1-tabname   = 'GT_FINAL'.
137    lv_fldcat1-fieldname = 'BATCH_SUM'.
138    lv_fldcat1-scrtext_m = 'Batch Amount'.
139    lv_fldcat1-outputlen = '15'.
140  *  lv_fldcat1-row_pos   = '1'.
141    lv_fldcat1-col_pos   = lv_index1.
142    APPEND lv_fldcat1 TO it_fcat1.
143
144  *  lv_index1  = lv_index1 + 1.
145  *  CLEAR lv_fldcat1.
146  *  lv_fldcat1-tabname   = 'GT_FINAL'.
147  *  lv_fldcat1-fieldname = 'MAX_PAY_AMT'.
148  *  lv_fldcat1-scrtext_m = 'Maximum payment'.
149  *  lv_fldcat1-outputlen = '15'.
150  **  lv_fldcat1-row_pos   = '1'.
151  *  lv_fldcat1-col_pos   = lv_index1.
152  *  APPEND lv_fldcat1 TO it_fcat1.
153
154    lv_index1  = lv_index1 + 1.
155    CLEAR lv_fldcat1.
156    lv_fldcat1-tabname   = 'GT_FINAL'.
157    lv_fldcat1-fieldname = 'CRUSR'.
158    lv_fldcat1-scrtext_m = 'Create User'.
159    lv_fldcat1-outputlen = '12'.
160  *  lv_fldcat1-row_pos   = '1'.
161    lv_fldcat1-col_pos   = lv_index1.
162    APPEND lv_fldcat1 TO it_fcat1.
163
164    lv_index1  = lv_index1 + 1.
165    CLEAR lv_fldcat1.
166    lv_fldcat1-tabname   = 'GT_FINAL'.
167    lv_fldcat1-fieldname = 'CRTIME'.
168    lv_fldcat1-scrtext_m = 'Create Time'.
169    lv_fldcat1-outputlen = '11'.
170    lv_fldcat1-row_pos   = '1'.
171    lv_fldcat1-col_pos   = lv_index1.
172    APPEND lv_fldcat1 TO it_fcat1.
173
174    lv_index1  = lv_index1 + 1.
175    CLEAR lv_fldcat1.
176    lv_fldcat1-tabname   = 'GT_FINAL'.
177    lv_fldcat1-fieldname = 'CRDATE'.
178    lv_fldcat1-scrtext_m = 'Create Date'.
```

```
179    lv_fldcat1-outputlen = '11'.
180  *  lv_fldcat1-row_pos   = '1'.
181    lv_fldcat1-col_pos   = lv_index1.
182    APPEND lv_fldcat1 TO it_fcat1.
183
184    lv_index1  = lv_index1 + 1.
185    CLEAR lv_fldcat1.
186    lv_fldcat1-tabname   = 'GT_FINAL'.
187    lv_fldcat1-fieldname = 'CHUSR'.
188    lv_fldcat1-scrtext_m = 'Change User'.
189    lv_fldcat1-outputlen = '12'.
190  *  lv_fldcat1-row_pos   = '1'.
191    lv_fldcat1-col_pos   = lv_index1.
192    APPEND lv_fldcat1 TO it_fcat1.
193
194    lv_index1  = lv_index1 + 1.
195    CLEAR lv_fldcat1.
196    lv_fldcat1-tabname   = 'GT_FINAL'.
197    lv_fldcat1-fieldname = 'CHTIME'.
198    lv_fldcat1-scrtext_m = 'Change Time'.
199    lv_fldcat1-outputlen = '11'.
200  *  lv_fldcat1-row_pos   = '1'.
201    lv_fldcat1-col_pos   = lv_index1.
202    APPEND lv_fldcat1 TO it_fcat1.
203
204    lv_index1  = lv_index1 + 1.
205    CLEAR lv_fldcat1.
206    lv_fldcat1-tabname   = 'GT_FINAL'.
207    lv_fldcat1-fieldname = 'CHDATE'.
208    lv_fldcat1-scrtext_m = 'Change Date'.
209    lv_fldcat1-outputlen = '11'.
210  *  lv_fldcat1-row_pos   = '1'.
211    lv_fldcat1-col_pos   = lv_index1.
212    APPEND lv_fldcat1 TO it_fcat1.
213
214    lv_index1  = lv_index1 + 1.
215    CLEAR lv_fldcat1.
216    lv_fldcat1-tabname   = 'GT_FINAL'.
217    lv_fldcat1-fieldname = 'ZBUKR'.
218    lv_fldcat1-scrtext_m = 'Paying company code'.
219    lv_fldcat1-outputlen = '19'.
220  *  lv_fldcat1-row_pos   = '1'.
221    lv_fldcat1-col_pos   = lv_index1.
222    APPEND lv_fldcat1 TO it_fcat1.
223
224  *  lv_index1  = lv_index1 + 1.
225  *   CLEAR lv_fldcat1.
226  *  lv_fldcat1-tabname   = 'GT_FINAL'.
227  *  lv_fldcat1-fieldname = 'HBKID'.
228  *  lv_fldcat1-scrtext_m = 'Short Key for a House Bank'.
229  *  lv_fldcat1-outputlen = '26'.
230  **  lv_fldcat1-row_pos   = '1'.
231  *  lv_fldcat1-col_pos   = lv_index1.
232  *  APPEND lv_fldcat1 TO it_fcat1.
233
234    lv_index1  = lv_index1 + 1.
235    CLEAR lv_fldcat1.
236    lv_fldcat1-tabname   = 'GT_FINAL'.
237    lv_fldcat1-fieldname = 'TOT_BTCH_AMT'.
238    lv_fldcat1-scrtext_m = 'Total batch amount'.
239    lv_fldcat1-outputlen = '18'.
```

```
240  *  lv_fldcat1-row_pos   = '1'.
241     lv_fldcat1-col_pos   = lv_index1.
242     APPEND lv_fldcat1 TO it_fcat1.
243  *
244  *  lv_index1  = lv_index1 + 1.
245  *  CLEAR lv_fldcat1.
246  *  lv_fldcat1-tabname   = 'GT_FINAL'.
247  *  lv_fldcat1-fieldname = 'MAXPAYAMT_RULECU'.
248  *  lv_fldcat1-scrtext_m = 'Maximum payment Amount'.
249  *  lv_fldcat1-outputlen = '22'.
250  *  lv_fldcat1-row_pos   = '1'.
251  *  lv_fldcat1-col_pos   = lv_index1.
252  *  APPEND lv_fldcat1 TO it_fcat1.
253
254  *   lv_index1  = lv_index1 + 1.
255  *   CLEAR lv_fldcat1.
256  *  lv_fldcat1-tabname   = 'GT_FINAL'.
257  *  lv_fldcat1-fieldname = 'GRP_FIELD1_VALUE'.
258  *  lv_fldcat1-scrtext_m = 'Grouping field val1'.
259  *  lv_fldcat1-outputlen = '30'.
260  *  lv_fldcat1-row_pos   = '1'.
261  *  lv_fldcat1-col_pos   = lv_index1.
262  *  APPEND lv_fldcat1 TO it_fcat1.
263  *
264  *   lv_index1  = lv_index1 + 1.
265  *   CLEAR lv_fldcat1.
266  *  lv_fldcat1-tabname   = 'GT_FINAL'.
267  *  lv_fldcat1-fieldname = 'GRP_FIELD2_VALUE'.
268  *  lv_fldcat1-scrtext_m = 'Grouping field val2'.
269  *  lv_fldcat1-outputlen = '30'.
270  *  lv_fldcat1-row_pos   = '1'.
271  *  lv_fldcat1-col_pos   = lv_index1.
272  *  APPEND lv_fldcat1 TO it_fcat1.
273
274  ENDFORM.                    " F_ALV_BUILD_FIELDCAT1
275  *&---------------------------------------------------------------------*
276  *&      Form  F_ALV_REPORT_LAYOUT1
277  *&---------------------------------------------------------------------*
278  *       text
279  *----------------------------------------------------------------------*
280  *  -->  p1        text
281  *  <--  p2        text
282  *----------------------------------------------------------------------*
283  FORM f_alv_report_layout1 .
284    CLEAR : it_layout1.
285    it_layout1-cwidth_opt = 'X'.
286    it_layout1-sel_mode   = 'A'.
287  ENDFORM.                    " F_ALV_REPORT_LAYOUT1
288  *&---------------------------------------------------------------------*
289  *&      Form  F_PREPARE_OP_TAB2
290  *&---------------------------------------------------------------------*
291  *       text
292  *----------------------------------------------------------------------*
293  *  -->  p1        text
294  *  <--  p2        text
295  *----------------------------------------------------------------------*
296  FORM f_prepare_op_tab2 .
297  *  BREAK sab_vaibhav.
298    REFRESH : gt_paym2, gt_batch_header2,gt_final2, gt_batch_sign.
299    SELECT * FROM zfi_paym_file INTO TABLE gt_paym2 WHERE sent = ' '.
300
```

```
301    DELETE gt_paym2 WHERE file_data_sent IS INITIAL.
302
303    IF gt_paym2 IS NOT INITIAL.
304      SELECT * FROM bnk_batch_header INTO TABLE gt_batch_header2
305                            FOR ALL ENTRIES IN gt_paym2
306                            WHERE laufi_f = gt_paym2-laufi
307                            AND   laufd_f = gt_paym2-laufd.
308
309      IF gt_batch_header2 IS NOT INITIAL.
310        SELECT * FROM  zfi_batch_sign INTO TABLE gt_batch_sign2
311                           FOR ALL ENTRIES IN gt_batch_header2
312                            WHERE batch_no = gt_batch_header2-batch_no
313                             AND DIGITL_SIGN = ' '
314                            AND   signer eq sy-uname
315                            and    snro  = '2'.
316
317  *sort gt_batch_sign by CDATE1 CTIME1.
318
319    LOOP AT gt_batch_header2 INTO gs_batch_header2.
320
321      READ TABLE gt_batch_sign2 INTO gs_batch_sign2 WITH KEY batch_no = gs
     _batch_header2-batch_no
322                                                            signer = sy-u
     name
323                                                            snro       =
     '2'.
324  *     IF sy-subrc <> 0.
325    IF sy-subrc EQ 0.
326        MOVE-CORRESPONDING gs_batch_header2 TO gs_final2.
327        READ TABLE gt_paym2 INTO gs_paym2 WITH KEY laufd = gs_batch_header
     2-laufd_f
328                                                   laufi = gs_batch_header
     2-laufi_f.
329        IF sy-subrc = 0.
330          gs_final2-raw_data = gs_paym2-raw_data.
331          gs_final2-file_data_sent = gs_paym2-file_data_sent.
332        ENDIF.
333        APPEND gs_final2 TO gt_final2.
334      ENDIF.
335  *  endif.
336      CLEAR : gs_batch_header2,gs_batch_sign2,gs_final2.
337    ENDLOOP.
338   ENDIF.
339    ENDIF.
340  ENDFORM.                    " F_PREPARE_OP_TAB2
341  *&---------------------------------------------------------------------*
342  *&      Form  F_ALV_BUILD_FIELDCAT2
343  *&---------------------------------------------------------------------*
344  *       text
345  *----------------------------------------------------------------------*
346  *  -->  p1        text
347  *  <--  p2        text
348  *----------------------------------------------------------------------*
349  FORM f_alv_build_fieldcat2 .
350    DATA lv_fldcat2 TYPE lvc_s_fcat.
351    DATA: lv_index2 TYPE sy-index.
352    REFRESH : it_fcat2.
353    CLEAR lv_fldcat2.
354    lv_fldcat2-tabname   = 'GT_FINAL2'.
355    lv_fldcat2-fieldname = 'BATCH_NO'.
356    lv_fldcat2-scrtext_m = 'Batch no.'.
```

```
357    lv_fldcat2-outputlen = '10'.
358  *  lv_fldcat2-row_pos   = '1'.
359    lv_fldcat2-col_pos   = lv_index2.
360    APPEND lv_fldcat2 TO it_fcat2.
361
362    lv_index2  = lv_index2 + 1.
363    CLEAR lv_fldcat2.
364    lv_fldcat2-tabname   = 'GT_FINAL2'.
365    lv_fldcat2-fieldname = 'RULE_ID'.
366    lv_fldcat2-scrtext_m = 'Rule Id'.
367    lv_fldcat2-outputlen = '10'.
368  *  lv_fldcat2-row_pos   = '1'.
369    lv_fldcat2-col_pos   = lv_index2.
370    APPEND lv_fldcat2 TO it_fcat2.
371
372    lv_index2  = lv_index2 + 1.
373    CLEAR lv_fldcat2.
374    lv_fldcat2-tabname   = 'GT_FINAL2'.
375    lv_fldcat2-fieldname = 'ITEM_CNT'.
376    lv_fldcat2-scrtext_m = 'No. of Payment'.
377    lv_fldcat2-outputlen = '14'.
378  *  lv_fldcat2-row_pos   = '1'.
379    lv_fldcat2-col_pos   = lv_index2.
380    APPEND lv_fldcat2 TO it_fcat2.
381
382    lv_index2  = lv_index2 + 1.
383    CLEAR lv_fldcat2.
384    lv_fldcat2-tabname   = 'GT_FINAL2'.
385    lv_fldcat2-fieldname = 'LAUFD'.
386    lv_fldcat2-scrtext_m = 'Merger date'.
387    lv_fldcat2-outputlen = '11'.
388  *  lv_fldcat2-row_pos   = '1'.
389    lv_fldcat2-col_pos   = lv_index2.
390    APPEND lv_fldcat2 TO it_fcat2.
391
392    lv_index2  = lv_index2 + 1.
393    CLEAR lv_fldcat2.
394    lv_fldcat2-tabname   = 'GT_FINAL2'.
395    lv_fldcat2-fieldname = 'LAUFI'.
396    lv_fldcat2-scrtext_m = 'Merge Id'.
397    lv_fldcat2-outputlen = '8'.
398  *  lv_fldcat2-row_pos   = '1'.
399    lv_fldcat2-col_pos   = lv_index2.
400    APPEND lv_fldcat2 TO it_fcat2.
401
402    lv_index2  = lv_index2 + 1.
403    CLEAR lv_fldcat2.
404    lv_fldcat2-tabname   = 'GT_FINAL2'.
405    lv_fldcat2-fieldname = 'LAUFD_F'.
406    lv_fldcat2-scrtext_m = 'File Date'.
407    lv_fldcat2-outputlen = '9'.
408  *  lv_fldcat2-row_pos   = '1'.
409    lv_fldcat2-col_pos   = lv_index2.
410    APPEND lv_fldcat2 TO it_fcat2.
411
412    lv_index2  = lv_index2 + 1.
413    CLEAR lv_fldcat2.
414    lv_fldcat2-tabname   = 'GT_FINAL2'.
415    lv_fldcat2-fieldname = 'LAUFI_F'.
416    lv_fldcat2-scrtext_m = 'File Id'.
417    lv_fldcat2-outputlen = '7'.
```

```
418  *  lv_fldcat2-row_pos   = '1'.
419     lv_fldcat2-col_pos   = lv_index2.
420     APPEND lv_fldcat2 TO it_fcat2.
421
422     lv_index2  = lv_index2 + 1.
423     CLEAR lv_fldcat2.
424     lv_fldcat2-tabname   = 'GT_FINAL2'.
425     lv_fldcat2-fieldname = 'BATCH_SUM'.
426     lv_fldcat2-scrtext_m = 'Batch Amount'.
427     lv_fldcat2-outputlen = '15'.
428  *  lv_fldcat2-row_pos   = '1'.
429     lv_fldcat2-col_pos   = lv_index2.
430     APPEND lv_fldcat2 TO it_fcat2.
431  *
432  *  lv_index2  = lv_index2 + 1.
433  *  CLEAR lv_fldcat2.
434  *  lv_fldcat2-tabname   = 'GT_FINAL2'.
435  *  lv_fldcat2-fieldname = 'MAX_PAY_AMT'.
436  *  lv_fldcat2-scrtext_m = 'Maximum payment'.
437  *  lv_fldcat2-outputlen = '15'.
438  **  lv_fldcat2-row_pos   = '1'.
439  *  lv_fldcat2-col_pos   = lv_index2.
440  *  APPEND lv_fldcat2 TO it_fcat2.
441
442     lv_index2  = lv_index2 + 1.
443     CLEAR lv_fldcat2.
444     lv_fldcat2-tabname   = 'GT_FINAL2'.
445     lv_fldcat2-fieldname = 'CRUSR'.
446     lv_fldcat2-scrtext_m = 'Create User'.
447     lv_fldcat2-outputlen = '12'.
448  *  lv_fldcat2-row_pos   = '1'.
449     lv_fldcat2-col_pos   = lv_index2.
450     APPEND lv_fldcat2 TO it_fcat2.
451
452     lv_index2  = lv_index2 + 1.
453     CLEAR lv_fldcat2.
454     lv_fldcat2-tabname   = 'GT_FINAL2'.
455     lv_fldcat2-fieldname = 'CRTIME'.
456     lv_fldcat2-scrtext_m = 'Create Time'.
457     lv_fldcat2-outputlen = '11'.
458     lv_fldcat2-row_pos   = '1'.
459     lv_fldcat2-col_pos   = lv_index2.
460     APPEND lv_fldcat2 TO it_fcat2.
461
462     lv_index2  = lv_index2 + 1.
463     CLEAR lv_fldcat2.
464     lv_fldcat2-tabname   = 'GT_FINAL2'.
465     lv_fldcat2-fieldname = 'CRDATE'.
466     lv_fldcat2-scrtext_m = 'Create Date'.
467     lv_fldcat2-outputlen = '11'.
468  *  lv_fldcat2-row_pos   = '1'.
469     lv_fldcat2-col_pos   = lv_index2.
470     APPEND lv_fldcat2 TO it_fcat2.
471
472     lv_index2  = lv_index2 + 1.
473     CLEAR lv_fldcat2.
474     lv_fldcat2-tabname   = 'GT_FINAL2'.
475     lv_fldcat2-fieldname = 'CHUSR'.
476     lv_fldcat2-scrtext_m = 'Change User'.
477     lv_fldcat2-outputlen = '12'.
478  *  lv_fldcat2-row_pos   = '1'.
```

```
479    lv_fldcat2-col_pos   = lv_index2.
480    APPEND lv_fldcat2 TO it_fcat2.
481
482    lv_index2  = lv_index2 + 1.
483    CLEAR lv_fldcat2.
484    lv_fldcat2-tabname   = 'GT_FINAL2'.
485    lv_fldcat2-fieldname = 'CHTIME'.
486    lv_fldcat2-scrtext_m = 'Change Time'.
487    lv_fldcat2-outputlen = '11'.
488  *  lv_fldcat2-row_pos   = '1'.
489    lv_fldcat2-col_pos   = lv_index2.
490    APPEND lv_fldcat2 TO it_fcat2.
491
492    lv_index2  = lv_index2 + 1.
493    CLEAR lv_fldcat2.
494    lv_fldcat2-tabname   = 'GT_FINAL2'.
495    lv_fldcat2-fieldname = 'CHDATE'.
496    lv_fldcat2-scrtext_m = 'Change Date'.
497    lv_fldcat2-outputlen = '11'.
498  *  lv_fldcat2-row_pos   = '1'.
499    lv_fldcat2-col_pos   = lv_index2.
500    APPEND lv_fldcat2 TO it_fcat2.
501
502    lv_index2  = lv_index2 + 1.
503    CLEAR lv_fldcat2.
504    lv_fldcat2-tabname   = 'GT_FINAL2'.
505    lv_fldcat2-fieldname = 'ZBUKR'.
506    lv_fldcat2-scrtext_m = 'Paying company code'.
507    lv_fldcat2-outputlen = '19'.
508  *  lv_fldcat2-row_pos   = '1'.
509    lv_fldcat2-col_pos   = lv_index2.
510    APPEND lv_fldcat2 TO it_fcat2.
511
512  *  lv_index2  = lv_index2 + 1.
513  *   CLEAR lv_fldcat2.
514  *  lv_fldcat2-tabname   = 'GT_FINAL2'.
515  *  lv_fldcat2-fieldname = 'HBKID'.
516  *  lv_fldcat2-scrtext_m = 'Short Key for a House Bank'.
517  *  lv_fldcat2-outputlen = '26'.
518  **  lv_fldcat2-row_pos   = '1'.
519  *  lv_fldcat2-col_pos   = lv_index2.
520  *  APPEND lv_fldcat2 TO it_fcat2.
521
522    lv_index2  = lv_index2 + 1.
523    CLEAR lv_fldcat2.
524    lv_fldcat2-tabname   = 'GT_FINAL2'.
525    lv_fldcat2-fieldname = 'TOT_BTCH_AMT'.
526    lv_fldcat2-scrtext_m = 'Total batch amount'.
527    lv_fldcat2-outputlen = '18'.
528  *  lv_fldcat2-row_pos   = '1'.
529    lv_fldcat2-col_pos   = lv_index2.
530    APPEND lv_fldcat2 TO it_fcat2.
531
532  *  lv_index2  = lv_index2 + 1.
533  *  CLEAR lv_fldcat2.
534  *  lv_fldcat2-tabname   = 'GT_FINAL2'.
535  *  lv_fldcat2-fieldname = 'MAXPAYAMT_RULECU'.
536  *  lv_fldcat2-scrtext_m = 'Maximum payment Amount'.
537  *  lv_fldcat2-outputlen = '22'.
538  *  lv_fldcat2-row_pos   = '1'.
539  *  lv_fldcat2-col_pos   = lv_index2.
```

```
*  APPEND lv_fldcat2 TO it_fcat2.

*   lv_index2  = lv_index2 + 1.
*   CLEAR lv_fldcat2.
*  lv_fldcat2-tabname   = 'GT_FINAL2'.
*  lv_fldcat2-fieldname = 'GRP_FIELD1_VALUE'.
*  lv_fldcat2-scrtext_m = 'Grouping field val1'.
*  lv_fldcat2-outputlen = '30'.
*  lv_fldcat2-row_pos   = '1'.
*  lv_fldcat2-col_pos   = lv_index2.
*  APPEND lv_fldcat2 TO it_fcat2.
*
*   lv_index2  = lv_index2 + 1.
*   CLEAR lv_fldcat2.
*  lv_fldcat2-tabname   = 'GT_FINAL2'.
*  lv_fldcat2-fieldname = 'GRP_FIELD2_VALUE'.
*  lv_fldcat2-scrtext_m = 'Grouping field val2'.
*  lv_fldcat2-outputlen = '30'.
*  lv_fldcat2-row_pos   = '1'.
*  lv_fldcat2-col_pos   = lv_index2.
*  APPEND lv_fldcat2 TO it_fcat2.


ENDFORM.                    " F_ALV_BUILD_FIELDCAT2
*&---------------------------------------------------------------------*
*&      Form  F_ALV_REPORT_LAYOUT2
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM f_alv_report_layout2 .
  CLEAR: it_layout2.
  it_layout2-cwidth_opt = 'X'.
  it_layout2-sel_mode   = 'A'.

ENDFORM.                    " F_ALV_REPORT_LAYOUT2
*&---------------------------------------------------------------------*
*&      Form  FREE_OBJECTS
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*  -->  p1        text
*  <--  p2        text
*----------------------------------------------------------------------*
FORM free_objects1 .

  CALL METHOD c_alvgd1->free
    EXCEPTIONS
      cntl_error        = 1
      cntl_system_error = 2
      OTHERS            = 3.
  IF sy-subrc <> 0.
    MESSAGE ID sy-msgid TYPE sy-msgty NUMBER sy-msgno
               WITH sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4.
  ENDIF.
  CALL METHOD c_ccont1->free
    EXCEPTIONS
      cntl_error        = 1
      cntl_system_error = 2
```

```
601         OTHERS           = 3.
602     IF sy-subrc <> 0.
603       MESSAGE ID sy-msgid TYPE sy-msgty NUMBER sy-msgno
604                  WITH sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4.
605     ENDIF.
606
607   ENDFORM.                    " FREE_OBJECTS
608   *&---------------------------------------------------------------------*
609   *&      Form  FREE_OBJECTS2
610   *&---------------------------------------------------------------------*
611   *       text
612   *----------------------------------------------------------------------*
613   *  -->  p1        text
614   *  <--  p2        text
615   *----------------------------------------------------------------------*
616   FORM free_objects2 .
617     CALL METHOD c_alvgd2->free
618       EXCEPTIONS
619         cntl_error        = 1
620         cntl_system_error = 2
621         OTHERS            = 3.
622     IF sy-subrc <> 0.
623       MESSAGE ID sy-msgid TYPE sy-msgty NUMBER sy-msgno
624                  WITH sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4.
625     ENDIF.
626     CALL METHOD c_ccont2->free
627       EXCEPTIONS
628         cntl_error        = 1
629         cntl_system_error = 2
630         OTHERS            = 3.
631     IF sy-subrc <> 0.
632       MESSAGE ID sy-msgid TYPE sy-msgty NUMBER sy-msgno
633                  WITH sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4.
634     ENDIF.
635
636
637   ENDFORM.                    " FREE_OBJECTS2
638   *&---------------------------------------------------------------------*
639   *&      Form  F_PREPARE_OP_TAB3
640   *&---------------------------------------------------------------------*
641   *       text
642   *----------------------------------------------------------------------*
643   *  -->  p1        text
644   *  <--  p2        text
645   *----------------------------------------------------------------------*
646   FORM f_prepare_op_tab3 .
647     REFRESH : gt_paym3, gt_batch_header3,gt_final3, gt_batch_sign3.
648   *break sab_vaibhav.
649     SELECT * FROM zfi_paym_file INTO TABLE gt_paym3 WHERE sent = 'X'.
650
651   *DELETE gt_paym3 WHERE file_data_sent IS INITIAL.
652
653     IF gt_paym3 IS NOT INITIAL.
654       SELECT * FROM bnk_batch_header INTO TABLE gt_batch_header3
655                              FOR ALL ENTRIES IN gt_paym3
656                              WHERE laufi_f = gt_paym3-laufi
657                              AND   laufd_f = gt_paym3-laufd.
658
659       IF gt_batch_header3 IS NOT INITIAL.
660         SELECT * FROM  zfi_batch_sign INTO TABLE gt_batch_sign3
661                           FOR ALL ENTRIES IN gt_batch_header3
```

```
662                         WHERE batch_no = gt_batch_header3-batch_no
663                         AND    signer EQ sy-uname.
664       ENDIF.
665     ENDIF.
666
667     LOOP AT gt_batch_header3 INTO gs_batch_header3.
668
669       READ TABLE gt_batch_sign3 INTO gs_batch_sign3 WITH KEY batch_no = gs
    _batch_header3-batch_no
670                                                              signer = sy-u
    name.
671       IF sy-subrc EQ 0.
672         MOVE-CORRESPONDING gs_batch_header3 TO gs_final3.
673         READ TABLE gt_paym3 INTO gs_paym3 WITH KEY laufd = gs_batch_header
    3-laufd_f
674                                                    laufi = gs_batch_header
    3-laufi_f.
675         IF sy-subrc = 0.
676           gs_final3-raw_data       = gs_paym3-raw_data.
677           gs_final3-file_data_sent = gs_paym3-file_data_sent.
678           gs_final3-error_flag     = gs_paym3-sent_error.
679         ENDIF.
680         APPEND gs_final3 TO gt_final3.
681       ENDIF.
682   *  endif.
683       CLEAR : gs_batch_header3,gs_batch_sign3,gs_final3.
684     ENDLOOP.
685
686   ENDFORM.                    " F_PREPARE_OP_TAB3
687
688   *&---------------------------------------------------------------------*
689   *&      Form  F_ALV_BUILD_FIELDCAT3
690   *&---------------------------------------------------------------------*
691   *       text
692   *----------------------------------------------------------------------*
693   *  -->  p1        text
694   *  <--  p2        text
695   *----------------------------------------------------------------------*
696   FORM f_alv_build_fieldcat3 .
697     DATA lv_fldcat3 TYPE lvc_s_fcat.
698     DATA: lv_index3 TYPE sy-index.
699     REFRESH : it_fcat3.
700     CLEAR lv_fldcat3.
701     lv_fldcat3-tabname   = 'GT_FINAL3'.
702     lv_fldcat3-fieldname = 'BATCH_NO'.
703     lv_fldcat3-scrtext_m = 'Batch no.'.
704     lv_fldcat3-outputlen = '10'.
705   *  lv_fldcat3-row_pos   = '1'.
706     lv_fldcat3-col_pos   = lv_index3.
707     APPEND lv_fldcat3 TO it_fcat3.
708
709     lv_index3  = lv_index3 + 1.
710     CLEAR lv_fldcat3.
711     lv_fldcat3-tabname   = 'GT_FINAL3'.
712     lv_fldcat3-fieldname = 'RULE_ID'.
713     lv_fldcat3-scrtext_m = 'Rule Id'.
714     lv_fldcat3-outputlen = '10'.
715   *  lv_fldcat3-row_pos   = '1'.
716     lv_fldcat3-col_pos   = lv_index3.
717     APPEND lv_fldcat3 TO it_fcat3.
718
```

```
719    lv_index3  = lv_index3 + 1.
720    CLEAR lv_fldcat3.
721    lv_fldcat3-tabname   = 'GT_FINAL3'.
722    lv_fldcat3-fieldname = 'ITEM_CNT'.
723    lv_fldcat3-scrtext_m = 'No. of Payment'.
724    lv_fldcat3-outputlen = '14'.
725  *  lv_fldcat3-row_pos   = '1'.
726    lv_fldcat3-col_pos   = lv_index3.
727    APPEND lv_fldcat3 TO it_fcat3.
728
729    lv_index3  = lv_index3 + 1.
730    CLEAR lv_fldcat3.
731    lv_fldcat3-tabname   = 'GT_FINAL3'.
732    lv_fldcat3-fieldname = 'LAUFD'.
733    lv_fldcat3-scrtext_m = 'Merger date'.
734    lv_fldcat3-outputlen = '11'.
735  *  lv_fldcat3-row_pos   = '1'.
736    lv_fldcat3-col_pos   = lv_index3.
737    APPEND lv_fldcat3 TO it_fcat3.
738
739    lv_index3  = lv_index3 + 1.
740    CLEAR lv_fldcat3.
741    lv_fldcat3-tabname   = 'GT_FINAL3'.
742    lv_fldcat3-fieldname = 'LAUFI'.
743    lv_fldcat3-scrtext_m = 'Merge Id'.
744    lv_fldcat3-outputlen = '8'.
745  *  lv_fldcat3-row_pos   = '1'.
746    lv_fldcat3-col_pos   = lv_index3.
747    APPEND lv_fldcat3 TO it_fcat3.
748
749    lv_index3  = lv_index3 + 1.
750    CLEAR lv_fldcat3.
751    lv_fldcat3-tabname   = 'GT_FINAL3'.
752    lv_fldcat3-fieldname = 'LAUFD_F'.
753    lv_fldcat3-scrtext_m = 'File Date'.
754    lv_fldcat3-outputlen = '9'.
755  *  lv_fldcat3-row_pos   = '1'.
756    lv_fldcat3-col_pos   = lv_index3.
757    APPEND lv_fldcat3 TO it_fcat3.
758
759    lv_index3  = lv_index3 + 1.
760    CLEAR lv_fldcat3.
761    lv_fldcat3-tabname   = 'GT_FINAL3'.
762    lv_fldcat3-fieldname = 'LAUFI_F'.
763    lv_fldcat3-scrtext_m = 'File Id'.
764    lv_fldcat3-outputlen = '7'.
765  *  lv_fldcat3-row_pos   = '1'.
766    lv_fldcat3-col_pos   = lv_index3.
767    APPEND lv_fldcat3 TO it_fcat3.
768
769
770    lv_index3  = lv_index3 + 1.
771    CLEAR lv_fldcat3.
772    lv_fldcat3-tabname   = 'GT_FINAL3'.
773    lv_fldcat3-fieldname = 'ERROR_FLAG'.
774    lv_fldcat3-scrtext_m = 'Error Flag'.
775    lv_fldcat3-outputlen = '10'.
776  *  lv_fldcat1-row_pos   = '1'.
777    lv_fldcat3-col_pos   = lv_index3.
778    APPEND lv_fldcat3 TO it_fcat3.
779
```

```
780    lv_index3  = lv_index3 + 1.
781    CLEAR lv_fldcat3.
782    lv_fldcat3-tabname   = 'GT_FINAL3'.
783    lv_fldcat3-fieldname = 'BATCH_SUM'.
784    lv_fldcat3-scrtext_m = 'Batch Amount'.
785    lv_fldcat3-outputlen = '15'.
786  *  lv_fldcat3-row_pos   = '1'.
787    lv_fldcat3-col_pos   = lv_index3.
788    APPEND lv_fldcat3 TO it_fcat3.
789  *
790  *  lv_index3  = lv_index3 + 1.
791  *  CLEAR lv_fldcat3.
792  *  lv_fldcat3-tabname   = 'GT_FINAL3'.
793  *  lv_fldcat3-fieldname = 'MAX_PAY_AMT'.
794  *  lv_fldcat3-scrtext_m = 'Maximum payment'.
795  *  lv_fldcat3-outputlen = '15'.
796  **  lv_fldcat3-row_pos   = '1'.
797  *  lv_fldcat3-col_pos   = lv_index3.
798  *  APPEND lv_fldcat3 TO it_fcat3.
799
800    lv_index3  = lv_index3 + 1.
801    CLEAR lv_fldcat3.
802    lv_fldcat3-tabname   = 'GT_FINAL3'.
803    lv_fldcat3-fieldname = 'CRUSR'.
804    lv_fldcat3-scrtext_m = 'Create User'.
805    lv_fldcat3-outputlen = '12'.
806  *  lv_fldcat3-row_pos   = '1'.
807    lv_fldcat3-col_pos   = lv_index3.
808    APPEND lv_fldcat3 TO it_fcat3.
809
810    lv_index3  = lv_index3 + 1.
811    CLEAR lv_fldcat3.
812    lv_fldcat3-tabname   = 'GT_FINAL3'.
813    lv_fldcat3-fieldname = 'CRTIME'.
814    lv_fldcat3-scrtext_m = 'Create Time'.
815    lv_fldcat3-outputlen = '11'.
816    lv_fldcat3-row_pos   = '1'.
817    lv_fldcat3-col_pos   = lv_index3.
818    APPEND lv_fldcat3 TO it_fcat3.
819
820    lv_index3  = lv_index3 + 1.
821    CLEAR lv_fldcat3.
822    lv_fldcat3-tabname   = 'GT_FINAL3'.
823    lv_fldcat3-fieldname = 'CRDATE'.
824    lv_fldcat3-scrtext_m = 'Create Date'.
825    lv_fldcat3-outputlen = '11'.
826  *  lv_fldcat3-row_pos   = '1'.
827    lv_fldcat3-col_pos   = lv_index3.
828    APPEND lv_fldcat3 TO it_fcat3.
829
830    lv_index3  = lv_index3 + 1.
831    CLEAR lv_fldcat3.
832    lv_fldcat3-tabname   = 'GT_FINAL3'.
833    lv_fldcat3-fieldname = 'CHUSR'.
834    lv_fldcat3-scrtext_m = 'Change User'.
835    lv_fldcat3-outputlen = '12'.
836  *  lv_fldcat3-row_pos   = '1'.
837    lv_fldcat3-col_pos   = lv_index3.
838    APPEND lv_fldcat3 TO it_fcat3.
839
840    lv_index3  = lv_index3 + 1.
```

```
841    CLEAR lv_fldcat3.
842    lv_fldcat3-tabname   = 'GT_FINAL3'.
843    lv_fldcat3-fieldname = 'CHTIME'.
844    lv_fldcat3-scrtext_m = 'Change Time'.
845    lv_fldcat3-outputlen = '11'.
846  *  lv_fldcat3-row_pos   = '1'.
847    lv_fldcat3-col_pos   = lv_index3.
848    APPEND lv_fldcat3 TO it_fcat3.
849
850    lv_index3  = lv_index3 + 1.
851    CLEAR lv_fldcat3.
852    lv_fldcat3-tabname   = 'GT_FINAL3'.
853    lv_fldcat3-fieldname = 'CHDATE'.
854    lv_fldcat3-scrtext_m = 'Change Date'.
855    lv_fldcat3-outputlen = '11'.
856  *  lv_fldcat3-row_pos   = '1'.
857    lv_fldcat3-col_pos   = lv_index3.
858    APPEND lv_fldcat3 TO it_fcat3.
859
860    lv_index3  = lv_index3 + 1.
861    CLEAR lv_fldcat3.
862    lv_fldcat3-tabname   = 'GT_FINAL3'.
863    lv_fldcat3-fieldname = 'ZBUKR'.
864    lv_fldcat3-scrtext_m = 'Paying company code'.
865    lv_fldcat3-outputlen = '19'.
866  *  lv_fldcat3-row_pos   = '1'.
867    lv_fldcat3-col_pos   = lv_index3.
868    APPEND lv_fldcat3 TO it_fcat3.
869
870  *  lv_index3  = lv_index3 + 1.
871  *   CLEAR lv_fldcat3.
872  *  lv_fldcat3-tabname   = 'GT_FINAL3'.
873  *  lv_fldcat3-fieldname = 'HBKID'.
874  *  lv_fldcat3-scrtext_m = 'Short Key for a House Bank'.
875  *  lv_fldcat3-outputlen = '26'.
876  **  lv_fldcat3-row_pos   = '1'.
877  *  lv_fldcat3-col_pos   = lv_index3.
878  *  APPEND lv_fldcat3 TO it_fcat3.
879
880    lv_index3  = lv_index3 + 1.
881    CLEAR lv_fldcat3.
882    lv_fldcat3-tabname   = 'GT_FINAL3'.
883    lv_fldcat3-fieldname = 'TOT_BTCH_AMT'.
884    lv_fldcat3-scrtext_m = 'Total batch amount'.
885    lv_fldcat3-outputlen = '18'.
886  *  lv_fldcat3-row_pos   = '1'.
887    lv_fldcat3-col_pos   = lv_index3.
888    APPEND lv_fldcat3 TO it_fcat3.
889
890  *  lv_index3  = lv_index3 + 1.
891  *  CLEAR lv_fldcat3.
892  *  lv_fldcat3-tabname   = 'GT_FINAL3'.
893  *  lv_fldcat3-fieldname = 'MAXPAYAMT_RULECU'.
894  *  lv_fldcat3-scrtext_m = 'Maximum payment Amount'.
895  *  lv_fldcat3-outputlen = '22'.
896  **  lv_fldcat3-row_pos   = '1'.
897  *  lv_fldcat3-col_pos   = lv_index3.
898  *  APPEND lv_fldcat3 TO it_fcat3.
899
900  *   lv_index3  = lv_index3 + 1.
901  *   CLEAR lv_fldcat3.
```

```
902  *  lv_fldcat3-tabname   = 'GT_FINAL3'.
903  *  lv_fldcat3-fieldname = 'GRP_FIELD1_VALUE'.
904  *  lv_fldcat3-scrtext_m = 'Grouping field val1'.
905  *  lv_fldcat3-outputlen = '30'.
906  **  lv_fldcat3-row_pos   = '1'.
907  *  lv_fldcat3-col_pos   = lv_index3.
908  *  APPEND lv_fldcat3 TO it_fcat3.
909  *
910  *   lv_index3  = lv_index3 + 1.
911  *   CLEAR lv_fldcat3.
912  *  lv_fldcat3-tabname   = 'GT_FINAL3'.
913  *  lv_fldcat3-fieldname = 'GRP_FIELD2_VALUE'.
914  *  lv_fldcat3-scrtext_m = 'Grouping field val2'.
915  *  lv_fldcat3-outputlen = '30'.
916  **  lv_fldcat3-row_pos   = '1'.
917  *  lv_fldcat3-col_pos   = lv_index3.
918  *  APPEND lv_fldcat3 TO it_fcat3.
919
920
921  ENDFORM.                    " F_ALV_BUILD_FIELDCAT3
922
923  *&---------------------------------------------------------------------*
924  *&      Form  F_ALV_REPORT_LAYOUT3
925  *&---------------------------------------------------------------------*
926  *       text
927  *----------------------------------------------------------------------*
928  *  -->  p1        text
929  *  <--  p2        text
930  *----------------------------------------------------------------------*
931  FORM f_alv_report_layout3 .
932    CLEAR: it_layout3.
933    it_layout3-cwidth_opt = 'X'.
934    it_layout3-sel_mode   = 'A'.
935  ENDFORM.                     " F_ALV_REPORT_LAYOUT3
936  *&---------------------------------------------------------------------*
937  *&      Form  DOWNLOAD_SENT_DATA
938  *&---------------------------------------------------------------------*
939  *       text
940  *----------------------------------------------------------------------*
941  *  -->  p1        text
942  *  <--  p2        text
943  *----------------------------------------------------------------------*
944  FORM download_sent_data .
945    DATA: lv_lines TYPE i.
946    DATA: fileleng TYPE i,
947          flen     TYPE i,
948          lv_filename TYPE string,
949          enh_version_management_hex_tb TYPE enh_version_management_hex_tb
     .
950
951    CALL METHOD c_alvgd3->get_selected_rows
952      IMPORTING
953        et_index_rows = gt_selected_rows3.
954
955    CLEAR : lv_lines.
956    DESCRIBE TABLE gt_selected_rows3 LINES lv_lines.
957    IF lv_lines EQ 0.
958      MESSAGE 'Please select one record' TYPE 'E'.
959    ELSEIF lv_lines > 1.
960      MESSAGE 'Please select only single record' TYPE 'E'.
961    ENDIF.
```

```
 962
 963    IF gt_selected_rows3 IS NOT INITIAL.
 964      READ TABLE gt_selected_rows3 INTO gs_selected_rows3 INDEX 1.
 965      IF sy-subrc = 0.
 966        READ TABLE gt_final3 INTO gs_final3 INDEX gs_selected_rows3-index.
 967        IF sy-subrc = 0.
 968          CLEAR gs_paym3.
 969          READ TABLE gt_paym3 INTO gs_paym3 WITH KEY laufi = gs_final3-lau
     fi_f
 970                                                     laufd = gs_final3-lau
     fd_f.
 971          IF sy-subrc EQ 0.
 972            CLEAR lv_filename.
 973            CONCATENATE 'C:\BCM\' gs_paym3-file_name '.TXT' INTO lv_filena
     me.
 974
 975            CALL FUNCTION 'ENH_XSTRING_TO_TAB'
 976              EXPORTING
 977                im_xstring = gs_final3-file_data_sent
 978              IMPORTING
 979                ex_data    = enh_version_management_hex_tb
 980                ex_leng    = flen.
 981
 982            CALL FUNCTION 'GUI_DOWNLOAD'
 983              EXPORTING
 984                bin_filesize            = flen        "ostr_updated_signed
     _data_l
 985                filename                = lv_filename   "'C:\TEST\TESTV1.T
     XT'
 986                filetype                = 'BIN'
 987  *             APPEND                  = ' '
 988  *             WRITE_FIELD_SEPARATOR   = ' '
 989  *             HEADER                  = '00'
 990  *             TRUNC_TRAILING_BLANKS   = ' '
 991              IMPORTING
 992                filelength              = fileleng
 993              TABLES
 994                data_tab                = enh_version_management_hex_tb  "
     OUT_DATA_TABLE
 995              EXCEPTIONS
 996                file_write_error        = 1
 997                no_batch                = 2
 998                gui_refuse_filetransfer = 3
 999                invalid_type            = 4
1000                no_authority            = 5
1001                unknown_error           = 6
1002                header_not_allowed      = 7
1003                separator_not_allowed   = 8
1004                filesize_not_allowed    = 9
1005                header_too_long         = 10
1006                dp_error_create         = 11
1007                dp_error_send           = 12
1008                dp_error_write          = 13
1009                unknown_dp_error        = 14
1010                access_denied           = 15
1011                dp_out_of_memory        = 16
1012                disk_full               = 17
1013                dp_timeout              = 18
1014                file_not_found          = 19
1015                dataprovider_exception  = 20
1016                control_flush_error     = 21
```

```
1017                OTHERS                  = 22.
1018              IF sy-subrc NE 0.
1019  *    MESSAGE E206(1S) WITH FILE.
1020              ENDIF.
1021              CLEAR :fileleng,flen.
1022              REFRESH : enh_version_management_hex_tb.
1023            ENDIF.
1024          ENDIF.
1025        ENDIF.
1026      ENDIF.
1027
1028  ENDFORM.                    " DOWNLOAD_SENT_DATA
1029
1030  *&---------------------------------------------------------------------*
1031  *&      Form  DOWNLOAD_RAW_DATA
1032  *&---------------------------------------------------------------------*
1033
1034  FORM download_raw_data .
1035    DATA : lv_lines TYPE i.
1036    DATA: fileleng TYPE i,
1037          flen     TYPE i,
1038          lv_filename TYPE string,
1039          enh_version_management_hex_tb TYPE enh_version_management_hex_tb
      .
1040
1041    CALL METHOD c_alvgd3->get_selected_rows
1042      IMPORTING
1043        et_index_rows = gt_selected_rows3.
1044
1045    CLEAR : lv_lines.
1046    DESCRIBE TABLE gt_selected_rows3 LINES lv_lines.
1047    IF lv_lines EQ 0.
1048      MESSAGE 'Please select one record' TYPE 'E'.
1049    ELSEIF lv_lines > 1.
1050      MESSAGE 'Please select only single record' TYPE 'E'.
1051    ENDIF.
1052
1053    IF gt_selected_rows3 IS NOT INITIAL.
1054      READ TABLE gt_selected_rows3 INTO gs_selected_rows3 INDEX 1.
1055      IF sy-subrc = 0.
1056        READ TABLE gt_final3 INTO gs_final3 INDEX gs_selected_rows3-index.
1057        IF sy-subrc = 0.
1058          CLEAR gs_paym3.
1059          READ TABLE gt_paym3 INTO gs_paym3 WITH KEY laufi = gs_final3-lau
      fi_f
1060                                                     laufd = gs_final3-lau
      fd_f.
1061          IF sy-subrc EQ 0.
1062            CLEAR lv_filename.
1063            CONCATENATE 'C:\BCM\' gs_paym3-file_name '.TXT' INTO lv_filena
      me.
1064
1065            CALL FUNCTION 'ENH_XSTRING_TO_TAB'
1066              EXPORTING
1067                im_xstring = gs_final3-raw_data
1068              IMPORTING
1069                ex_data    = enh_version_management_hex_tb
1070                ex_leng    = flen.
1071
1072            CALL FUNCTION 'GUI_DOWNLOAD'
1073              EXPORTING
```

```
1074                bin_filesize            = flen        "ostr_updated_signed
      _data_l
1075                filename                = lv_filename   "'C:\TEST\TESTV1.T
      XT'
1076                filetype                = 'BIN'
1077  *             APPEND                  = ' '
1078  *             WRITE_FIELD_SEPARATOR   = ' '
1079  *             HEADER                  = '00'
1080  *             TRUNC_TRAILING_BLANKS   = ' '
1081              IMPORTING
1082                filelength              = fileleng
1083              TABLES
1084                data_tab                = enh_version_management_hex_tb  "
      OUT_DATA_TABLE
1085              EXCEPTIONS
1086                file_write_error        = 1
1087                no_batch                = 2
1088                gui_refuse_filetransfer = 3
1089                invalid_type            = 4
1090                no_authority            = 5
1091                unknown_error           = 6
1092                header_not_allowed      = 7
1093                separator_not_allowed   = 8
1094                filesize_not_allowed    = 9
1095                header_too_long         = 10
1096                dp_error_create         = 11
1097                dp_error_send           = 12
1098                dp_error_write          = 13
1099                unknown_dp_error        = 14
1100                access_denied           = 15
1101                dp_out_of_memory        = 16
1102                disk_full               = 17
1103                dp_timeout              = 18
1104                file_not_found          = 19
1105                dataprovider_exception  = 20
1106                control_flush_error     = 21
1107                OTHERS                  = 22.
1108            IF sy-subrc NE 0.
1109  *     MESSAGE E206(1S) WITH FILE.
1110            ENDIF.
1111            CLEAR : fileleng,flen.
1112            REFRESH : enh_version_management_hex_tb.
1113          ENDIF.
1114        ENDIF.
1115      ENDIF.
1116    ENDIF.
1117
1118  ENDFORM.                    " DOWNLOAD_RAW_DATA
1119  *&---------------------------------------------------------------------*
1120  *&      Form  FREE_OBJECTS3
1121  *&---------------------------------------------------------------------*
1122  *       text
1123  *----------------------------------------------------------------------*
1124  *  -->  p1        text
1125  *  <--  p2        text
1126  *----------------------------------------------------------------------*
1127  FORM free_objects3 .
1128    CALL METHOD c_alvgd3->free
1129      EXCEPTIONS
1130        cntl_error        = 1
1131        cntl_system_error = 2
```

```
1132        OTHERS            = 3.
1133    IF sy-subrc <> 0.
1134      MESSAGE ID sy-msgid TYPE sy-msgty NUMBER sy-msgno
1135                 WITH sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4.
1136    ENDIF.
1137    CALL METHOD c_ccont3->free
1138      EXCEPTIONS
1139        cntl_error        = 1
1140        cntl_system_error = 2
1141        OTHERS            = 3.
1142    IF sy-subrc <> 0.
1143      MESSAGE ID sy-msgid TYPE sy-msgty NUMBER sy-msgno
1144                 WITH sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4.
1145    ENDIF.
1146
1147
1148  ENDFORM.                    " FREE_OBJECTS3
```

```
*&---------------------------------------------------------------------*
*&  Include           ZFI_BNK_APP1_I01
*&---------------------------------------------------------------------*

*&SPWIZARD: INPUT MODULE FOR TS 'TABSTRIP'. DO NOT CHANGE THIS LINE!
*&SPWIZARD: GETS ACTIVE TAB
MODULE tabstrip_active_tab_get INPUT.
*  ok_code = sy-ucomm.
*  CASE ok_code.
*    WHEN c_tabstrip-tab1.
*      g_tabstrip-pressed_tab = c_tabstrip-tab1.
*    WHEN c_tabstrip-tab2.
*      g_tabstrip-pressed_tab = c_tabstrip-tab2.
*    WHEN c_tabstrip-tab3.
*      g_tabstrip-pressed_tab = c_tabstrip-tab3.
*    WHEN OTHERS.
**&SPWIZARD:      DO NOTHING
*  ENDCASE.
ENDMODULE.                    "TABSTRIP_ACTIVE_TAB_GET INPUT
*&---------------------------------------------------------------------*
*&      Module  GET_SELECTED_ROW_TAB1  INPUT
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
MODULE get_selected_row_tab1 INPUT.
  DATA : ok_code1 TYPE sy-ucomm.
  ok_code1 = sy-ucomm.
  DATA: crc      TYPE i.
  DATA: doc_sig  TYPE xstring.
  DATA: lv_filename TYPE string.

  DATA: sig_list TYPE ssfsignertab.
  DATA: signer   TYPE ssfsigner.
  DATA: doc_ver  TYPE xstring.
  DATA: doc_line TYPE string.
  DATA: doc_tab  TYPE ssftxttab.
  DATA l_sign TYPE ssfsigner.
  DATA it_zuser_signer TYPE TABLE OF zuser_signer.
  DATA wa_zuser_signer TYPE zuser_signer.

  DATA:     ev_owner           TYPE string,
            ev_email           TYPE string,
            ev_serial          TYPE string,
            ev_thumbprint      TYPE string,
            ev_validfrom       TYPE string,
            ev_validto         TYPE string,
            ev_issuer          TYPE string,
            ev_bindocument_out TYPE xstring,
            e_int              TYPE i.

  DATA : lv_lines TYPE i.

  DATA: go_file_verifier        TYPE REF TO lcl_file_verifier.

  IF ok_code1 = 'APPROVE1'.

***        Getting the selected rows index
    CALL METHOD c_alvgd1->get_selected_rows
      IMPORTING
        et_index_rows = gt_selected_rows1.

```

```
62      CLEAR : lv_lines.
63      DESCRIBE TABLE gt_selected_rows1 LINES lv_lines.
64      IF lv_lines EQ 0.
65        MESSAGE 'Please select one record' TYPE 'I'.
66        EXIT.
67      ELSEIF lv_lines > 1.
68        MESSAGE 'Please select only single record' TYPE 'I'.
69        EXIT.
70      ENDIF.
71
72      IF gt_selected_rows1 IS NOT INITIAL.
73        READ TABLE gt_selected_rows1 INTO gs_selected_rows1 INDEX 1.
74        IF sy-subrc = 0.
75  " Code Remediation changes S4 2025_1_A Conversion **BEGIN OF CHANGE BY S
    AP_ABAP 06.06.2026  FOR ATC
76          READ TABLE gt_final INTO gs_final INDEX gs_selected_rows1-index.
     "#EC CI_NOORDER
77  " Code Remediation changes S4 2025_1_A Conversion * *END OF CHANGE BY SA
    P_ABAP 06.06.2026  FOR ATC
78          IF sy-subrc = 0.
79            CALL FUNCTION 'SSFS_CALL_CONTROL'
80              EXPORTING
81  *              TXTDOCUMENT      = doc_tab
82                bindocument      = gs_final-raw_data "doc_bin
83                doc_type         = 'TXT'
84  *              ssf_id           = 'CN=Kashyap Banaam Bhushan, SP=Delhi, p
    ostalCode=110092, OU=CID - 2923089, O=Oil and Natural Gas Corporation Lt
    d., C=IN'
85              IMPORTING
86                crc              = crc
87                signature        = doc_sig
88              EXCEPTIONS
89                parameter_error  = 1
90                conversion_error = 2
91                control_error    = 3
92                frontend_error   = 4
93                OTHERS           = 5.
94
95
96            IF crc = 0.
97              REFRESH sig_list.
98              CALL FUNCTION 'SSFS_SERVER_VERIFY'
99                EXPORTING
100 *                SSFAPPLIC       =
101                 signature       = doc_sig
102               IMPORTING
103                 crc             = crc
104                 signerlist      = sig_list
105                 txtdocument_out = doc_tab
106                 bindocument_out = doc_ver
107               EXCEPTIONS
108                 kernel_error    = 1
109                 parameter_error = 2
110                 OTHERS          = 3.
111             IF crc = 0.
112               go_file_verifier = lcl_file_verifier=>get_instance( ).
113               READ TABLE sig_list INTO l_sign INDEX 1.
114               IF sy-subrc = 0.
115                 CALL METHOD go_file_verifier->get_signer_info
116                   EXPORTING
117                     is_signerlist = l_sign
```

```
118                   IMPORTING
119                     ev_owner      = ev_owner
120                     ev_email      = ev_email
121                     ev_serial     = ev_serial
122                     ev_thumbprint = ev_thumbprint
123                     ev_validfrom  = ev_validfrom
124                     ev_validto    = ev_validto
125                     ev_issuer     = ev_issuer
126                     e_int         = e_int.
127                 ENDIF.
128                 SELECT * INTO TABLE it_zuser_signer
129                   FROM zuser_signer WHERE uname = sy-uname
130                                       AND active = 'X'.
131                 IF sy-subrc = 0.
132                   READ TABLE it_zuser_signer INTO wa_zuser_signer WITH KEY
     thumbprin = ev_thumbprint.
133                   IF sy-subrc <> 0.
134                     MESSAGE 'Wrong Signature used for signing' TYPE 'I'.
135                     EXIT.
136                   ENDIF.
137                 ELSE.
138                   MESSAGE 'No Signature Maintained for user' TYPE 'I'.
139                   EXIT.
140                 ENDIF.
141               ENDIF.
142
143               CLEAR gs_paym.
144               READ TABLE gt_paym INTO gs_paym WITH KEY laufi = gs_final-la
    ufi_f
145                                                       laufd = gs_final-la
    ufd_f.
146               IF sy-subrc = 0.
147                 gs_paym-file_data_sent = doc_sig.
148                 MODIFY zfi_paym_file FROM gs_paym.
149               ENDIF.
150
151
152   ***** modify record in table ZFI_BATCH_SIGN
153               CLEAR : gs_batch_sign.
154               READ TABLE gt_batch_sign INTO gs_batch_sign WITH KEY batch_n
    o = gs_final-batch_no
155                                                                  signer
     = sy-uname.
156               IF sy-subrc = 0.
157                 gs_batch_sign-batch_no    = gs_final-batch_no.
158                 gs_batch_sign-signer      = sy-uname.
159                 gs_batch_sign-digitl_sign = 'X'.
160                 gs_batch_sign-cdate1      = sy-datum.
161                 gs_batch_sign-ctime1      = sy-uzeit.
162                 MODIFY zfi_batch_sign FROM gs_batch_sign.
163                 REFRESH : gt_selected_rows1.
164               ENDIF.
165             ELSE.
166               CASE crc.
167                 WHEN '2'.
168                   MESSAGE 'Please insert E-token' TYPE 'I'.
169                   EXIT.
170               ENDCASE.
171             ENDIF.
172
173           ENDIF.
```

```
174       ENDIF.
175     ENDIF.
176   ENDIF.
177 ENDMODULE.                    " GET_SELECTED_ROW_TAB1  INPUT
178 *&---------------------------------------------------------------------*
179 *&      Module  GET_SELECTED_ROW_TAB2  INPUT
180 *&---------------------------------------------------------------------*
181 *       text
182 *---------------------------------------------------------------------*
183 MODULE get_selected_row_tab2 INPUT.
184   REFRESH : gt_selected_rows2.
185   DATA: ok_code2  TYPE sy-ucomm,
186         lv_lines1 TYPE i.
187
188   ok_code2 = sy-ucomm.
189   DATA: lv_crc   TYPE i.
190   DATA: lv_doc_sig  TYPE xstring.
191
192   DATA: lo_bnk_detail    TYPE REF TO zco_sios_bank_interface_encryp, "zc
    o_sios_sbiwebservice_encrypt,  "zfi_bnk_sbi_in_encco_si_sbiweb, 14.07.20
    16
193         proxy_data       TYPE zfile_upload2,                        "z
    fi_bnk_sbi_in_encfile_upload2, 14.07.2016
194         lt_input         TYPE zfile_upload_response2,               "z
    fi_bnk_sbi_in_encfile_upload1, 14.07.2016
195         lo_sys_exception TYPE REF TO cx_ai_system_fault.
196   DATA:  err_string       TYPE string.
197
198   DATA: sig_list1 TYPE ssfsignertab.
199   DATA: signer1   TYPE ssfsigner.
200   DATA: doc_ver1  TYPE xstring.
201   DATA: doc_line1 TYPE string.
202   DATA: doc_tab1  TYPE ssftxttab.
203   DATA l_sign1 TYPE ssfsigner.
204   DATA it_zuser_signer1 TYPE TABLE OF zuser_signer.
205   DATA wa_zuser_signer1 TYPE zuser_signer.
206
207   DATA:     ev_owner1           TYPE string,
208             ev_email1           TYPE string,
209             ev_serial1          TYPE string,
210             ev_thumbprint1      TYPE string,
211             ev_validfrom1       TYPE string,
212             ev_validto1         TYPE string,
213             ev_issuer1          TYPE string,
214             ev_bindocument_out1 TYPE xstring,
215             e_int1              TYPE i.
216
217   DATA: go_file_verifier1        TYPE REF TO lcl_file_verifier.
218
219   CREATE OBJECT lo_bnk_detail.
220
221
222   IF ok_code2 = 'APPROVE2'.
223     CALL METHOD c_alvgd2->get_selected_rows
224       IMPORTING
225         et_index_rows = gt_selected_rows2.
226
227     CLEAR : lv_lines1.
228     DESCRIBE TABLE gt_selected_rows2 LINES lv_lines1.
229     IF lv_lines1 EQ 0.
230       MESSAGE 'Please select one record' TYPE 'I'.
```

```
231         EXIT.
232       ELSEIF lv_lines1 > 1.
233         MESSAGE 'Please select only single record' TYPE 'I'.
234         EXIT.
235       ENDIF.
236
237       IF gt_selected_rows2 IS NOT INITIAL.
238         READ TABLE gt_selected_rows2 INTO gs_selected_rows2 INDEX 1.
239         IF sy-subrc = 0.
240           READ TABLE gt_final2 INTO gs_final2 INDEX gs_selected_rows2-inde
    x.
241           IF sy-subrc = 0.
242             CALL FUNCTION 'SSFS_CALL_CONTROL'
243               EXPORTING
244 *               TXTDOCUMENT      = doc_tab
245                 bindocument      = gs_final2-file_data_sent "doc_bin
246                 doc_type         = 'TXT'
247 *               ssf_id           = 'CN=Kashyap Banaam Bhushan, SP=Delhi, p
    ostalCode=110092, OU=CID - 2923089, O=Oil and Natural Gas Corporation Lt
    d., C=IN'
248               IMPORTING
249                 crc              = lv_crc
250                 signature        = lv_doc_sig
251               EXCEPTIONS
252                 parameter_error  = 1
253                 conversion_error = 2
254                 control_error    = 3
255                 frontend_error   = 4
256                 OTHERS           = 5.
257
258
259             IF lv_crc = 0.
260               REFRESH sig_list1.
261               CALL FUNCTION 'SSFS_SERVER_VERIFY'
262                 EXPORTING
263 *                 SSFAPPLIC       =
264                   signature       = lv_doc_sig
265                 IMPORTING
266                   crc             = lv_crc
267                   signerlist      = sig_list1
268                   txtdocument_out = doc_tab1
269                   bindocument_out = doc_ver1
270                 EXCEPTIONS
271                   kernel_error    = 1
272                   parameter_error = 2
273                   OTHERS          = 3.
274 *BREAK-POINT.
275               IF lv_crc = 0.
276                 go_file_verifier1 = lcl_file_verifier=>get_instance( ).
277
278                 READ TABLE sig_list1 INTO l_sign1 INDEX 1.
279                 IF sy-subrc = 0.
280                   CALL METHOD go_file_verifier1->get_signer_info
281                     EXPORTING
282                       is_signerlist = l_sign1
283                     IMPORTING
284                       ev_owner      = ev_owner1
285                       ev_email      = ev_email1
286                       ev_serial     = ev_serial1
287                       ev_thumbprint = ev_thumbprint1
288                       ev_validfrom  = ev_validfrom1
```

```
289                     ev_validto    = ev_validto1
290                     ev_issuer     = ev_issuer1
291                     e_int         = e_int1.
292                 ENDIF.
293
294                 SELECT * INTO TABLE it_zuser_signer1
295                   FROM zuser_signer WHERE  uname = sy-uname
296                                        AND active = 'X'.
297                 IF sy-subrc = 0.
298                   READ TABLE it_zuser_signer1 INTO wa_zuser_signer1 WITH K
    EY thumbprin = ev_thumbprint1.
299                   IF sy-subrc <> 0.
300                     MESSAGE 'Wrong Signature used for signing' TYPE 'I'.
301                     EXIT.
302                   ENDIF.
303                 ELSE.
304                   MESSAGE 'No Signature Maintained for user' TYPE 'I'.
305                   EXIT.
306                 ENDIF.
307               ENDIF.
308
309               CLEAR gs_paym2.
310               READ TABLE gt_paym2 INTO gs_paym2 WITH KEY laufi = gs_final2
    -laufi_f
311                                                          laufd = gs_final2
    -laufd_f.
312               IF sy-subrc = 0.
313                 IF gs_paym2-sent <> ' '.
314                   MESSAGE 'Can not be sent' TYPE 'I'.
315                   EXIT.
316                 ELSE.
317                   proxy_data-parameters-arg0  = gs_paym2-file_name.
318                   proxy_data-parameters-arg1  = lv_doc_sig.
319                   TRY.
320                       CALL METHOD lo_bnk_detail->sios_bank_interface_encry
    ption  "sios_sbiwebservice_encryption           "si_sbiwebservice_enc
321                         EXPORTING
322                           output = proxy_data
323                         IMPORTING
324                           input  = lt_input.
325                     CATCH cx_ai_system_fault INTO lo_sys_exception.
326                       err_string = lo_sys_exception->get_text( ).
327
328                     CATCH cx_ai_application_fault .
329                   ENDTRY.
330                   IF err_string IS NOT INITIAL.
331                     gs_paym2-file_data_sent = lv_doc_sig.
332                     gs_paym2-sent_error     = 'X'.
333                     gs_paym2-sent           = 'X'.
334                     gs_paym2-sent_date      = sy-datum.
335                     gs_paym2-sent_time      = sy-uzeit.
336                     MODIFY zfi_paym_file FROM gs_paym2.
337                     MESSAGE 'Error in sending' TYPE 'I'.
338                   ELSE.
339                     gs_paym2-file_data_sent = lv_doc_sig.
340                     gs_paym2-sent           = 'X'.
341                     gs_paym2-sent_date      = sy-datum.
342                     gs_paym2-sent_time      = sy-uzeit.
343                     MODIFY zfi_paym_file FROM gs_paym2.
344                   ENDIF.
345  ***** insert record in table ZFI_BATCH_SIGN
```

```
346                 CLEAR : gs_batch_sign2.
347                 READ TABLE gt_batch_sign2 INTO gs_batch_sign2 WITH KEY b
    atch_no = gs_final2-batch_no
348                                                                       s
    igner   = sy-uname.
349                 IF sy-subrc = 0.
350                   gs_batch_sign2-batch_no    = gs_final2-batch_no.
351                   gs_batch_sign2-signer      = sy-uname.
352                   gs_batch_sign2-digitl_sign = 'X'.
353                   gs_batch_sign2-cdate1      = sy-datum.
354                   gs_batch_sign2-ctime1      = sy-uzeit.
355                   MODIFY zfi_batch_sign FROM gs_batch_sign2.
356                   CLEAR : proxy_data.
357                 ENDIF.
358               ENDIF.
359             ENDIF.
360           ENDIF.
361         ENDIF.
362       ENDIF.
363     ENDIF.
364   ENDIF.
365 ENDMODULE.                    " GET_SELECTED_ROW_TAB2  INPUT
366 *&---------------------------------------------------------------------*
367 *&      Module  USER_COMMAND_0110  INPUT
368 *&---------------------------------------------------------------------*
369 *       text
370 *---------------------------------------------------------------------*
371 MODULE user_command_0101 INPUT.
372   DATA lv_ucomm TYPE sy-ucomm.
373   lv_ucomm = sy-ucomm.
374   CASE lv_ucomm.
375     WHEN 'APPROVE1'.
376       PERFORM free_objects1.
377     WHEN 'CANCEL' OR 'EXIT'.
378       PERFORM free_objects1.
379       LEAVE PROGRAM.
380     WHEN 'BACK'.
381       PERFORM free_objects1.
382 *      SET SCREEN '0'.
383       LEAVE PROGRAM.
384   ENDCASE.
385
386
387 ENDMODULE.                    " USER_COMMAND_0110  INPUT
388 *&---------------------------------------------------------------------*
389 *&      Module  USER_COMMAND_0102  INPUT
390 *&---------------------------------------------------------------------*
391 *       text
392 *---------------------------------------------------------------------*
393 MODULE user_command_0102 INPUT.
394   DATA lv_ucomm1 TYPE sy-ucomm.
395   lv_ucomm1 = sy-ucomm.
396   CASE lv_ucomm1.
397     WHEN 'APPROVE2'.
398       PERFORM free_objects2.
399     WHEN 'CANCEL' OR 'EXIT'.
400       PERFORM free_objects2.
401       LEAVE PROGRAM.
402     WHEN 'BACK'.
403       PERFORM free_objects2.
404 *      SET SCREEN '0'.
```

```
405         LEAVE PROGRAM.
406     ENDCASE.
407   ENDMODULE.                    " USER_COMMAND_0102  INPUT
408   *&---------------------------------------------------------------------*
409   *&      Module  USER_COMMAND_0100  INPUT
410   *&---------------------------------------------------------------------*
411   *       text
412   *---------------------------------------------------------------------*
413   MODULE user_command_0100 INPUT.
414   *DATA lv_ucomm2 TYPE sy-ucomm.
415   *  lv_ucomm2 = sy-ucomm.
416   *  CASE lv_ucomm2.
417   *    WHEN 'CANCEL' OR 'EXIT'.
418   *      PERFORM free_objects2.
419   *      LEAVE PROGRAM.
420   *    WHEN 'BACK'.
421   *      PERFORM free_objects2.
422   *      SET SCREEN '0'.
423   *      LEAVE SCREEN.
424   *  ENDCASE.
425   ENDMODULE.                    " USER_COMMAND_0100  INPUT
426
427   *&---------------------------------------------------------------------*
428   *&      Module  GET_SELECTED_ROW_TAB3  INPUT
429   *&---------------------------------------------------------------------*
430   *       text
431   *---------------------------------------------------------------------*
432   MODULE get_selected_row_tab3 INPUT.
433     DATA : lv_code TYPE sy-ucomm.
434     lv_code = sy-ucomm.
435
436     DATA: fileleng                      TYPE i,
437           flen                          TYPE i,
438           enh_version_management_hex_tb TYPE enh_version_management_hex_tb
      .
439
440     IF lv_code = 'DOWNLOAD1'.
441       PERFORM download_sent_data.
442     ELSEIF lv_code =  'DOWNLOAD2'.
443       PERFORM download_raw_data.
444     ENDIF.
445
446   ENDMODULE.                    " GET_SELECTED_ROW_TAB3  INPUT
447   *&---------------------------------------------------------------------*
448   *&      Module  USER_COMMAND_0103  INPUT
449   *&---------------------------------------------------------------------*
450   *       text
451   *---------------------------------------------------------------------*
452   MODULE user_command_0103 INPUT.
453     DATA lv_ucomm3 TYPE sy-ucomm.
454     lv_ucomm3 = sy-ucomm.
455     CASE lv_ucomm3.
456       WHEN 'DOWNLOAD1' OR 'DOWNLOAD2'.
457   *      PERFORM free_objects3.
458       WHEN 'CANCEL' OR 'EXIT'.
459   *      PERFORM free_objects3.
460         LEAVE PROGRAM.
461       WHEN 'BACK'.
462   *      PERFORM free_objects3.
463         LEAVE PROGRAM.
464     ENDCASE.
```

```
465  ENDMODULE.                    " USER_COMMAND_0103  INPUT
```

```
*&---------------------------------------------------------------------*
*&  Include           ZFI_BNK_APP1_O01
*&---------------------------------------------------------------------*

*&SPWIZARD: OUTPUT MODULE FOR TS 'TABSTRIP'. DO NOT CHANGE THIS LINE!
*&SPWIZARD: SETS ACTIVE TAB
MODULE tabstrip_active_tab_set OUTPUT.
*  tabstrip-activetab = g_tabstrip-pressed_tab.
*  CASE g_tabstrip-pressed_tab.
*    WHEN c_tabstrip-tab1.
*      g_tabstrip-subscreen = '0101'.
*    WHEN c_tabstrip-tab2.
*      g_tabstrip-subscreen = '0102'.
*    WHEN c_tabstrip-tab3.
*      g_tabstrip-subscreen = '0103'.
*    WHEN OTHERS.
**&SPWIZARD:      DO NOTHING
*  ENDCASE.
ENDMODULE.                    "TABSTRIP_ACTIVE_TAB_SET OUTPUT
*&---------------------------------------------------------------------*
*&      Module  OUTPUT_TAB1  OUTPUT
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
MODULE output_tab1 OUTPUT.
  PERFORM f_prepare_op_tab1.
***
  CREATE OBJECT c_ccont1
    EXPORTING
      container_name = 'CONTROL1'.

  CREATE OBJECT c_alvgd1
    EXPORTING
      i_parent = c_ccont1.

**Set field for ALV
  PERFORM f_alv_build_fieldcat1.

**Set ALV attributes FOR LAYOUT
  PERFORM f_alv_report_layout1.

  CHECK NOT c_alvgd1 IS INITIAL.

**Call ALV GRID
  CALL METHOD c_alvgd1->set_table_for_first_display
    EXPORTING
      is_layout                     = it_layout1
    CHANGING
      it_outtab                     = gt_final
      it_fieldcatalog               = it_fcat1
    EXCEPTIONS
      invalid_parameter_combination = 1
      program_error                 = 2
      too_many_lines                = 3
      OTHERS                        = 4.
  IF sy-subrc <> 0.
    MESSAGE ID sy-msgid TYPE sy-msgty NUMBER sy-msgno
               WITH sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4.
  ENDIF.

ENDMODULE.                 " OUTPUT_TAB1  OUTPUT
```

```
*&---------------------------------------------------------------------*
*&      Module  OUTPUT_TAB2  OUTPUT
*&---------------------------------------------------------------------*
*       text
*---------------------------------------------------------------------*
MODULE output_tab2 OUTPUT.
  PERFORM f_prepare_op_tab2.
***
  CREATE OBJECT c_ccont2
    EXPORTING
      container_name = 'CONTROL2'.

  CREATE OBJECT c_alvgd2
    EXPORTING
      i_parent = c_ccont2.

*  **Set field for ALV
  PERFORM f_alv_build_fieldcat2.

**Set ALV attributes FOR LAYOUT
  PERFORM f_alv_report_layout2.

  CHECK NOT c_alvgd2 IS INITIAL.

**Call ALV GRID
  CALL METHOD c_alvgd2->set_table_for_first_display
    EXPORTING
      is_layout                     = it_layout2
    CHANGING
      it_outtab                     = gt_final2
      it_fieldcatalog               = it_fcat2
    EXCEPTIONS
      invalid_parameter_combination = 1
      program_error                 = 2
      too_many_lines                = 3
      OTHERS                        = 4.
  IF sy-subrc <> 0.
    MESSAGE ID sy-msgid TYPE sy-msgty NUMBER sy-msgno
               WITH sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4.
  ENDIF.

ENDMODULE.                 " OUTPUT_TAB2  OUTPUT
*&---------------------------------------------------------------------*
*&      Module  STATUS_0100  OUTPUT
*&---------------------------------------------------------------------*
*       text
*---------------------------------------------------------------------*
MODULE status_0100 OUTPUT.

  SET PF-STATUS 'ZPF_FI_BNK_APP1'.
  SET TITLEBAR 'ZTITLE_FI_BNK_APP1'.

ENDMODULE.                 " STATUS_0100  OUTPUT


*&---------------------------------------------------------------------*
*&      Module  OUTPUT_TAB3  OUTPUT
*&---------------------------------------------------------------------*
*       text
*---------------------------------------------------------------------*
MODULE output_tab3 OUTPUT.
```

```
123     PERFORM f_prepare_op_tab3.
124   ***
125     IF  c_ccont3 IS INITIAL.
126
127       CREATE OBJECT c_ccont3
128         EXPORTING
129           container_name = 'CONTROL3'.
130     ENDIF.
131
132     IF c_alvgd3 IS INITIAL.
133       CREATE OBJECT c_alvgd3
134         EXPORTING
135           i_parent = c_ccont3.
136     ENDIF.
137   *  **Set field for ALV
138     PERFORM f_alv_build_fieldcat3.
139
140   **Set ALV attributes FOR LAYOUT
141     PERFORM f_alv_report_layout3.
142
143     CHECK NOT c_alvgd3 IS INITIAL.
144
145   **Call ALV GRID
146     CALL METHOD c_alvgd3->set_table_for_first_display
147       EXPORTING
148         is_layout                     = it_layout3
149       CHANGING
150         it_outtab                     = gt_final3
151         it_fieldcatalog               = it_fcat3
152       EXCEPTIONS
153         invalid_parameter_combination = 1
154         program_error                 = 2
155         too_many_lines                = 3
156         OTHERS                        = 4.
157     IF sy-subrc EQ 0.
158   *    CALL METHOD c_alvgd3->refresh_table_display.
159
160     ENDIF.
161   ENDMODULE.                 " OUTPUT_TAB3  OUTPUT
```

```
 1  *&---------------------------------------------------------------------*
 2  *& Include ZFI_BNK_APP1_TOP                                 Module Pool
        ZFI_BNK_APP1
 3  *&
 4  *&---------------------------------------------------------------------*
 5
 6  PROGRAM ZFI_BNK_APP1.
 7
 8  TYPES : BEGIN OF ty_final,
 9          sel              type c,
10          guid             TYPE bnk_com_btch_guid,
11          batch_no         TYPE bnk_com_btch_no,
12          rule_id          TYPE bnk_com_btch_rule_id,
13          item_cnt         TYPE bnk_com_btch_ctr,
14          laufd            TYPE bnk_com_btch_mrge_dat,
15          laufi            TYPE bnk_com_btch_mrge_id,
16          xvorl            TYPE xvorl,
17        laufd_f            TYPE bnk_com_btch_file_dat,
18        laufi_f            TYPE bnk_com_btch_file_id,
19        Error_flag         type c,
20        batch_sum          TYPE bnk_com_btch_amount,
21        batch_curr         TYPE bnk_com_btch_curr,
22        max_pay_amt        TYPE bnk_com_max_paymnt_amount,
23        status             TYPE bnk_com_btch_status_id,
24        crusr              TYPE bnk_com_create_user,
25        crtime             TYPE bnk_com_create_time,
26        crdate             TYPE bnk_com_create_date,
27        chusr              TYPE bnk_com_change_user,
28        chtime             TYPE bnk_com_change_time,
29        chdate             TYPE bnk_com_change_date,
30        cur_processor      TYPE bnk_com_cur_processor,
31        archive_status     TYPE bnk_com_archive_status,
32        zbukr              TYPE dzbukr,
33        hbkid              TYPE hbkid,
34        tot_btch_amt       TYPE bnk_com_btch_amt_in_rule_curr,
35        maxpayamt_rulecu TYPE bnk_com_max_pymntamt_in_rulcur,
36        grp_field1_value TYPE bnk_com_grp_fld_val1,
37        grp_field2_value TYPE bnk_com_grp_fld_val2,
38        raw_data           TYPE XSTRING,
39        file_data_sent     TYPE XSTRING,
40
41  end of ty_final.
42
43  DATA : gt_paym           TYPE STANDARD TABLE OF zfi_paym_file,
44         gt_batch_header   TYPE STANDARD TABLE OF bnk_batch_header,
45         gt_batch_sign     TYPE STANDARD TABLE OF zfi_batch_sign ,
46         gt_paym2          TYPE STANDARD TABLE OF zfi_paym_file,
47         gt_batch_header2 TYPE STANDARD TABLE OF bnk_batch_header,
48         gt_batch_sign2    TYPE STANDARD TABLE OF zfi_batch_sign ,
49         gt_paym3          TYPE STANDARD TABLE OF zfi_paym_file,
50         gt_batch_header3 TYPE STANDARD TABLE OF bnk_batch_header,
51         gt_batch_sign3    TYPE STANDARD TABLE OF zfi_batch_sign ,
52         gt_final          TYPE STANDARD TABLE OF ty_final,
53         gt_final2         TYPE STANDARD TABLE OF ty_final,
54         gt_final3         type STANDARD TABLE OF ty_final.
55
56  DATA: gs_paym           LIKE LINE OF gt_paym,
57        gs_batch_header   LIKE LINE OF gt_batch_header,
58        gs_batch_sign     LIKE LINE OF gt_batch_sign,
59        gs_final          LIKE LINE OF gt_final,
60        gs_paym2          LIKE LINE OF gt_paym2,
```

```
       gs_batch_header2  LIKE LINE OF gt_batch_header2,
       gs_batch_sign2    LIKE LINE OF gt_batch_sign2,
       gs_final2         LIKE LINE OF gt_final2,
       gs_paym3          LIKE LINE OF gt_paym3,
       gs_batch_header3  LIKE LINE OF gt_batch_header3,
       gs_batch_sign3    LIKE LINE OF gt_batch_sign3,
       gs_final3         type ty_final.


*&SPWIZARD: FUNCTION CODES FOR TABSTRIP 'TABSTRIP'
CONSTANTS: BEGIN OF C_TABSTRIP,
             TAB1 LIKE SY-UCOMM VALUE 'TABSTRIP_FC1',
             TAB2 LIKE SY-UCOMM VALUE 'TABSTRIP_FC2',
             TAB3 LIKE SY-UCOMM VALUE 'TABSTRIP_FC3',
           END OF C_TABSTRIP.
*&SPWIZARD: DATA FOR TABSTRIP 'TABSTRIP'
CONTROLS:  TABSTRIP TYPE TABSTRIP.
DATA:      BEGIN OF G_TABSTRIP,
             SUBSCREEN   LIKE SY-DYNNR,
             PROG        LIKE SY-REPID VALUE 'ZFI_BNK_APP1',
             PRESSED_TAB LIKE SY-UCOMM VALUE C_TABSTRIP-TAB1,
           END OF G_TABSTRIP.
DATA:      OK_CODE LIKE SY-UCOMM.

DATA: c_ccont1         TYPE REF TO cl_gui_custom_container,   "Custom co
nt
      c_alvgd1         TYPE REF TO cl_gui_alv_grid,            "ALV grid
      it_fcat1         TYPE lvc_t_fcat,                        " Fieldca
t
      it_layout1       TYPE lvc_s_layo.                         "Layout

DATA: c_ccont2         TYPE REF TO cl_gui_custom_container,   "Custom co
nt
      c_alvgd2         TYPE REF TO cl_gui_alv_grid,   "ALV grid
      it_fcat2         TYPE lvc_t_fcat,             " Fieldcat
      it_layout2       TYPE lvc_s_layo.

DATA: c_ccont3         TYPE REF TO cl_gui_custom_container,   "Custom co
nt
      c_alvgd3         TYPE REF TO cl_gui_alv_grid,   "ALV grid
      it_fcat3         TYPE lvc_t_fcat,             " Fieldcat
      it_layout3       TYPE lvc_s_layo.

data: gt_selected_rows1 type lvc_t_row,"Selected Rows
      gs_selected_rows1 type lvc_s_row.

data: gt_selected_rows2 type lvc_t_row,"Selected Rows
      gs_selected_rows2 type lvc_s_row.

data: gt_selected_rows3 type lvc_t_row,"Selected Rows
      gs_selected_rows3 type lvc_s_row.
```

Title:
Program ZFI_BNK_APP1

Owner..... SAP
Short text..... Texte der allgemeinen Basis
Edit Language DE
Status 21.05.26

00EXT

Owner..... SAP
Short text..... Allg. Basis: z.B. Benutzer und Berecht.pflege
Edit Language DE
Status 21.05.26

01EXT

PAI modules:		Line	Page
GET_SELECTED_ROW_TAB1.....	ZFI_BNK_APP1_I01	25	29
GET_SELECTED_ROW_TAB2.....	ZFI_BNK_APP1_I01	183	32
GET_SELECTED_ROW_TAB3.....	ZFI_BNK_APP1_I01	432	36
TABSTRIP_ACTIVE_TAB_GET.....	ZFI_BNK_APP1_I01	7	29
USER_COMMAND_0100.....	ZFI_BNK_APP1_I01	413	36
USER_COMMAND_0101.....	ZFI_BNK_APP1_I01	371	35
USER_COMMAND_0102.....	ZFI_BNK_APP1_I01	393	35
USER_COMMAND_0103.....	ZFI_BNK_APP1_I01	452	36

PBO modules:		Line	Page
OUTPUT_TAB1.....	ZFI_BNK_APP1_O01	25	38
OUTPUT_TAB2.....	ZFI_BNK_APP1_O01	67	39
OUTPUT_TAB3.....	ZFI_BNK_APP1_O01	122	39
STATUS_0100.....	ZFI_BNK_APP1_O01	109	39
TABSTRIP_ACTIVE_TAB_SET.....	ZFI_BNK_APP1_O01	7	38

Subroutines:		Line	Page
DOWNLOAD_RAW_DATA.....	ZFI_BNK_APP1_F01	1034	26
	ZFI_BNK_APP1_I01	443	36
DOWNLOAD_SENT_DATA.....	ZFI_BNK_APP1_F01	944	24
	ZFI_BNK_APP1_I01	441	36
FREE_OBJECTS1.....	ZFI_BNK_APP1_F01	586	18
	ZFI_BNK_APP1_I01	376	35
	ZFI_BNK_APP1_I01	378	35
	ZFI_BNK_APP1_I01	381	35
FREE_OBJECTS2.....	ZFI_BNK_APP1_F01	616	19
	ZFI_BNK_APP1_I01	398	35
	ZFI_BNK_APP1_I01	400	35
	ZFI_BNK_APP1_I01	403	35
FREE_OBJECTS3.....	ZFI_BNK_APP1_F01	1127	27
F_ALV_BUILD_FIELDCAT1.....	ZFI_BNK_APP1_F01	61	10
	ZFI_BNK_APP1_O01	37	38
F_ALV_BUILD_FIELDCAT2.....	ZFI_BNK_APP1_F01	349	14
	ZFI_BNK_APP1_O01	79	39
F_ALV_BUILD_FIELDCAT3.....	ZFI_BNK_APP1_F01	696	20
	ZFI_BNK_APP1_O01	138	40
F_ALV_REPORT_LAYOUT1.....	ZFI_BNK_APP1_F01	283	13
	ZFI_BNK_APP1_O01	40	38
F_ALV_REPORT_LAYOUT2.....	ZFI_BNK_APP1_F01	572	18
	ZFI_BNK_APP1_O01	82	39
F_ALV_REPORT_LAYOUT3.....	ZFI_BNK_APP1_F01	931	24
	ZFI_BNK_APP1_O01	141	40
F_PREPARE_OP_TAB1.....	ZFI_BNK_APP1_F01	8	9
	ZFI_BNK_APP1_O01	26	38
F_PREPARE_OP_TAB2.....	ZFI_BNK_APP1_F01	296	13
	ZFI_BNK_APP1_O01	68	39
F_PREPARE_OP_TAB3.....	ZFI_BNK_APP1_F01	646	19
	ZFI_BNK_APP1_O01	123	40

Status:		Line	Page
ZPF_FI_BNK_APP1.....	ZFI_BNK_APP1_O01	111	39

Messages:

		Line	Page
?.....	ZBCM_CLASS_DECLARE	169	6
?.....	ZBCM_CLASS_DECLARE	173	6
?.....	ZBCM_CLASS_DECLARE	176	6
?.....	ZFI_BNK_APP1_F01	594	18
?.....	ZFI_BNK_APP1_F01	603	19
?.....	ZFI_BNK_APP1_F01	623	19
?.....	ZFI_BNK_APP1_F01	632	19
?.....	ZFI_BNK_APP1_F01	958	24
?.....	ZFI_BNK_APP1_F01	960	24
?.....	ZFI_BNK_APP1_F01	1048	26
?.....	ZFI_BNK_APP1_F01	1050	26
?.....	ZFI_BNK_APP1_F01	1134	28
?.....	ZFI_BNK_APP1_F01	1143	28
?.....	ZFI_BNK_APP1_I01	65	30
?.....	ZFI_BNK_APP1_I01	68	30
?.....	ZFI_BNK_APP1_I01	134	31
?.....	ZFI_BNK_APP1_I01	138	31
?.....	ZFI_BNK_APP1_I01	168	31
?.....	ZFI_BNK_APP1_I01	230	32
?.....	ZFI_BNK_APP1_I01	233	33
?.....	ZFI_BNK_APP1_I01	300	34
?.....	ZFI_BNK_APP1_I01	304	34
?.....	ZFI_BNK_APP1_I01	314	34
?.....	ZFI_BNK_APP1_I01	337	34
?.....	ZFI_BNK_APP1_O01	57	38
?.....	ZFI_BNK_APP1_O01	99	39

External calls

Function Modules		Line	Page
ENH_XSTRING_TO_TAB.....	ZFI_BNK_APP1_F01	975	25
	ZFI_BNK_APP1_F01	1065	26
GUI_DOWNLOAD.....	ZFI_BNK_APP1_F01	982	25
	ZFI_BNK_APP1_F01	1072	26
SSFS_CALL_CONTROL.....	ZFI_BNK_APP1_I01	79	30
	ZFI_BNK_APP1_I01	242	33
SSFS_SERVER_VERIFY.....	ZBCM_CLASS_DECLARE	154	6
	ZFI_BNK_APP1_I01	98	30
	ZFI_BNK_APP1_I01	261	33
SX_INTERNET_ADDRESS_TO_NORMAL.	ZBCM_CLASS_DECLARE	272	8

DATABASE OPERATIONS

SELECT:

		Line	Page
BNK_BATCH_HEADER.....	ZFI_BNK_APP1_F01	17	9
	ZFI_BNK_APP1_F01	304	14
	ZFI_BNK_APP1_F01	654	19
ZFI_BATCH_SIGN.....	ZFI_BNK_APP1_F01	23	9
	ZFI_BNK_APP1_F01	310	14
	ZFI_BNK_APP1_F01	660	19
ZFI_PAYM_FILE.....	ZFI_BNK_APP1_F01	11	9
	ZFI_BNK_APP1_F01	299	13
	ZFI_BNK_APP1_F01	649	19
ZUSER_SIGNER.....	ZFI_BNK_APP1_I01	129	31
	ZFI_BNK_APP1_I01	295	34

MODIFY:

		Line	Page
ZFI_BATCH_SIGN.....	ZFI_BNK_APP1_I01	162	31
	ZFI_BNK_APP1_I01	355	35
ZFI_PAYM_FILE.....	ZFI_BNK_APP1_I01	148	31
	ZFI_BNK_APP1_I01	336	34
	ZFI_BNK_APP1_I01	343	34

INTERNAL TABLE OPERATIONS

READ TABLE:

		Line	Page
GT_BATCH_SIGN.....	ZFI_BNK_APP1_F01	32	9
	ZFI_BNK_APP1_I01	154	31
GT_BATCH_SIGN2.....	ZFI_BNK_APP1_F01	321	14
	ZFI_BNK_APP1_I01	347	35
GT_BATCH_SIGN3.....	ZFI_BNK_APP1_F01	669	20
GT_FINAL.....	ZFI_BNK_APP1_I01	76	30
GT_FINAL2.....	ZFI_BNK_APP1_I01	240	33
GT_FINAL3.....	ZFI_BNK_APP1_F01	966	25
	ZFI_BNK_APP1_F01	1056	26
GT_PAYM.....	ZFI_BNK_APP1_F01	38	9
	ZFI_BNK_APP1_I01	144	31
GT_PAYM2.....	ZFI_BNK_APP1_F01	327	14
	ZFI_BNK_APP1_I01	310	34
GT_PAYM3.....	ZFI_BNK_APP1_F01	673	20
	ZFI_BNK_APP1_F01	969	25
	ZFI_BNK_APP1_F01	1059	26
GT_SELECTED_ROWS1.....	ZFI_BNK_APP1_I01	73	30
GT_SELECTED_ROWS2.....	ZFI_BNK_APP1_I01	238	33
GT_SELECTED_ROWS3.....	ZFI_BNK_APP1_F01	964	25
	ZFI_BNK_APP1_F01	1054	26
IT_ZUSER_SIGNER.....	ZFI_BNK_APP1_I01	132	31
IT_ZUSER_SIGNER1.....	ZFI_BNK_APP1_I01	298	34
LT_SIGNERLIST.....	ZBCM_CLASS_DECLARE	178	6
SIG_LIST.....	ZFI_BNK_APP1_I01	113	30
SIG_LIST1.....	ZFI_BNK_APP1_I01	278	33

DELETE:

		Line	Page
GT_PAYM.....	ZFI_BNK_APP1_F01	13	9
	ZFI_BNK_APP1_F01	14	9
GT_PAYM2.....	ZFI_BNK_APP1_F01	301	14